Lab 3 - Explaining Deep Learning Models

How the lab works

Authors: Ariful Islam Riane (3981886) and Pedro Fernandez Tonso (3983781) - **Group number:** 19

- The questions are divided into three levels.
  - **[Basic]**: if your group completes all Basic questions of a lab correctly, you get a 6 (a passing grade).
  - **[Regular]**: these questions give up to 2 extra points.
  - **[Advanced]**: for students looking for a challenge; expect to work on them in your own time.
- Every group works with slightly different data and settings, so your answers, numbers and plots are different from those of other groups.
  - Set your group number G in the first code cell. Your seed is $S = G$; use it everywhere a random seed is needed (`random_state=S`).
  - *Your patient* is row $p = S \bmod n$ of your test set, where n is the number of rows in the test set.
  - Put your group number in the title of every plot.
- Answers must use the numbers and plots from your own notebook. An answer that does not match your own output gets no points.
- You may use AI tools to help with code, but every group member must be able to explain what you hand in.
- Lines marked `# TODO` must be completed. Write your answers in the **Your answers** cells.
- Hand in this notebook with all cells run. Also convert the notebook with all cells run into PDF and hand in.

Data and models. A pretrained ImageNet model (ResNet-50) and the images in `images/` ; Fashion-MNIST; MNIST with a small CNN. Your group's image, Fashion-MNIST classes, digits and mystery model are given by `assignment(GROUP_NUMBER)` . The first run downloads the pretrained model and the data sets, which takes a few minutes.

Setup

In [1]:
```
import copy
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

import torch
import torchvision.transforms as T
from torchvision import datasets
from torchvision.models import resnet50, ResNet50_Weights
```

```
from captum.attr import Occlusion, Saliency, IntegratedGradients
from captum.attr import visualization as viz
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
from pytorch_grad_cam.utils.image import show_cam_on_image
import shap

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.linear_model import LogisticRegression
from scipy.stats import spearmanr

from xai_lab_utils import (group_seed, assignment, FASHION_CLASSES, accuracy,
                           load_or_train_mnist_cnn, mnist_loaders,
load_mystery_model)
```

In [2]:
```
# Fill in your group number. Everything else depends on it.
GROUP_NUMBER = 19  # TODO: your group number, e.g. 7

S = group_seed(GROUP_NUMBER)   # use S everywhere a random seed is needed
print("Group", GROUP_NUMBER, "- seed S =", S)
torch.manual_seed(S)

a = assignment(GROUP_NUMBER)
a
```

```
Group 19 - seed S = 19
```

Out[2]:
```
{'fashion_pair': (4, 6),
 'fashion_pair_names': ('Coat', 'Shirt'),
 'digit_pair': (3, 5),
 'mystery_id': 3,
 'image': 'images\\stingray.jpg'}
```

In [3]:
```
# Pretrained ImageNet model
weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights).eval()
preprocess = weights.transforms()
labels = weights.meta["categories"]
resize_crop = T.Compose([T.Resize(preprocess.resize_size),
T.CenterCrop(preprocess.crop_size)])


def load_image(path):
    """Returns the RGB image (224x224, values in [0, 1]) and the model input
tensor."""
    img = Image.open(path).convert("RGB")
    rgb = np.asarray(resize_crop(img), dtype=np.float32) / 255
    x = preprocess(img).unsqueeze(0)
    return rgb, x


def top_k(x, k=5):
    with torch.no_grad():
        prob = model(x).softmax(1)[0]
    values, idx = prob.topk(k)
    for v, i in zip(values, idx):
        print(f"{labels[i]:30s} {v:.3f}")
    return idx.tolist()
```

Q1) Occlusion

a) **[Basic]** Use the code below to find the areas of importance for your assigned image and one other image from the directory. A running example is given for a basketball image. Use the heatmaps to explain qualitatively how well the pretrained model understands the object in the image.

b) **[Basic]** Explain what is important in the data set, so that what we as humans intuitively use to identify objects can be learnt by the machine. Hint: refer back to the lecture example of a misclassification.

c) **[Advanced]** Change the size of the occlusion window (16, 32, 64 pixels). How do the heatmap and the run time change?

d) **[Advanced]** Occlude with a black patch, a grey patch and a blurred patch. How does the heatmap change? Why does the choice of the patch matter?

> **Hint:** `Occlusion(model).attribute(x, target=idx, sliding_window_shapes=(3, 32, 32), strides=(3, 16, 16), baselines=0)` ; occlusion is slow on a laptop without a GPU; keep the window at 32 and the stride at 16

In [4]:
```
EXAMPLE_IMAGE = "images/basketball.jpg"   # running example

rgb, x = load_image(EXAMPLE_IMAGE)
idx = top_k(x)[0]

occlusion = Occlusion(model)
attr = occlusion.attribute(x, target=idx, sliding_window_shapes=(3, 32, 32),
                           strides=(3, 16, 16), baselines=0)
attr = np.transpose(attr.squeeze(0).detach().numpy(), (1, 2, 0))

viz.visualize_image_attr(attr, rgb, method="blended_heat_map", sign="positive",
                         show_colorbar=True, title=f"Occlusion: {labels[idx]} -
group {GROUP_NUMBER}")
```

```
basketball                    0.245
spindle                       0.006
rugby ball                    0.004
golf ball                     0.003
puck                          0.002
```



Out[4]:
```
(<Figure size 600x600 with 2 Axes>,
 <Axes: title={'center': 'Occlusion: basketball - group 19'}>)
```

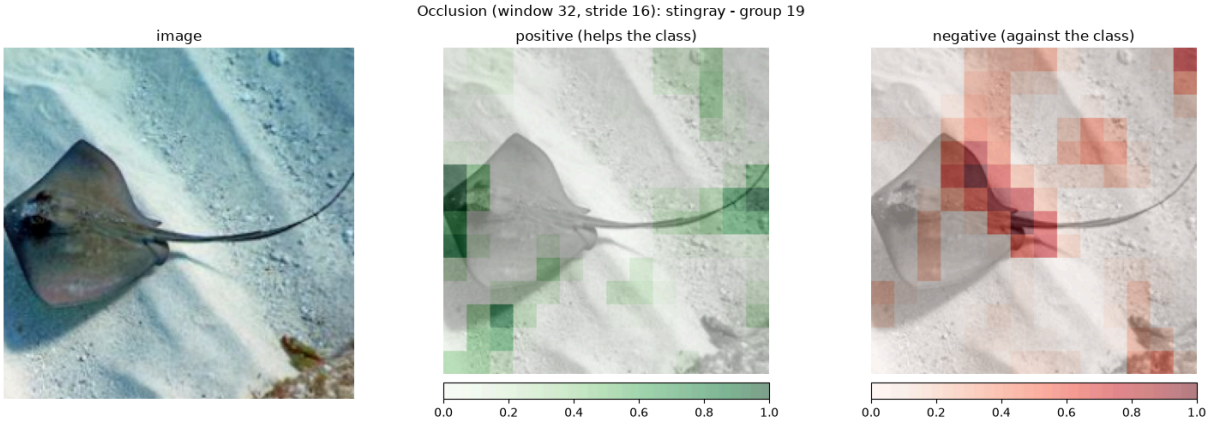
In [5]:
```
# (a) our assigned image and one other image (a dog and a ram: two objects in one
picture)
def occlusion_map(x, idx, window=32, stride=16, baselines=0):
    attr = Occlusion(model).attribute(x, target=idx, sliding_window_shapes=(3,
window, window),
                                      strides=(3, stride, stride),
baselines=baselines,
                                      perturbations_per_eval=16)
    return np.transpose(attr.squeeze(0).detach().numpy(), (1, 2, 0))


def show_occlusion(path, idx=None):
    rgb, x = load_image(path)
    print(path)
    top = top_k(x)
    idx = top[0] if idx is None else idx
    attr = occlusion_map(x, idx)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes[0].imshow(rgb)
    axes[0].set_title("image")
    viz.visualize_image_attr(attr, rgb, method="blended_heat_map",
sign="positive", show_colorbar=True,
                             plt_fig_axis=(fig, axes[1]), use_pyplot=False,
title="positive (helps the class)")
    viz.visualize_image_attr(attr, rgb, method="blended_heat_map",
sign="negative", show_colorbar=True,
```

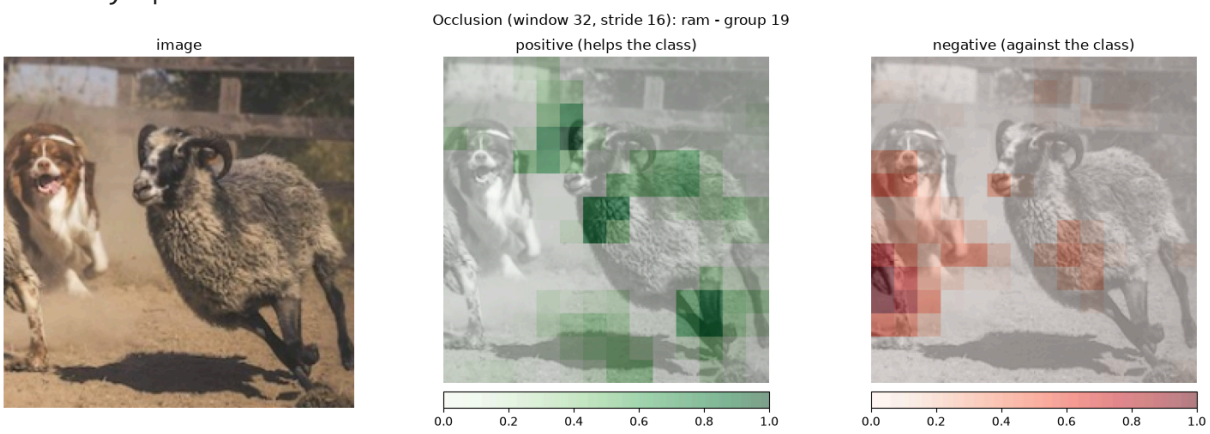
```
                                plt_fig_axis=(fig, axes[2]), use_pyplot=False,
title="negative (against the class)")
    for ax in axes:
        ax.axis("off")
    fig.suptitle(f"Occlusion (window 32, stride 16): {labels[idx]} - group
{GROUP_NUMBER}")
    plt.tight_layout()
    plt.show()
    return attr


OUR_IMAGE = a["image"]
attr_ours = show_occlusion(OUR_IMAGE)
attr_other = show_occlusion("images/dog_and_ram.jpg")
```

```
images\stingray.jpg
stingray                    0.251
electric ray                0.203
harvestman                  0.002
shovel                      0.002
cockroach                   0.002
```



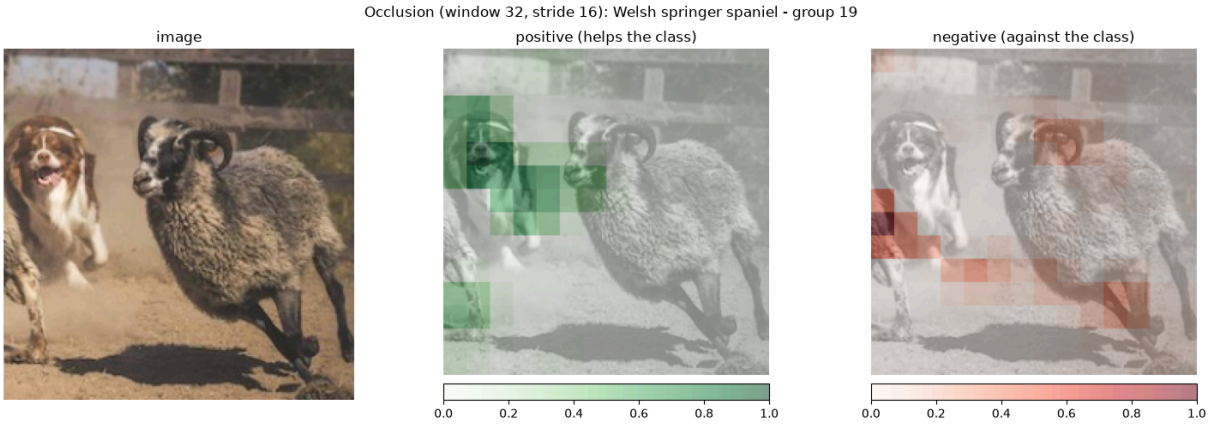

```
images/dog_and_ram.jpg
ram                         0.047
Welsh springer spaniel      0.034
Border collie               0.027
llama                       0.023
Brittany spaniel            0.020
```



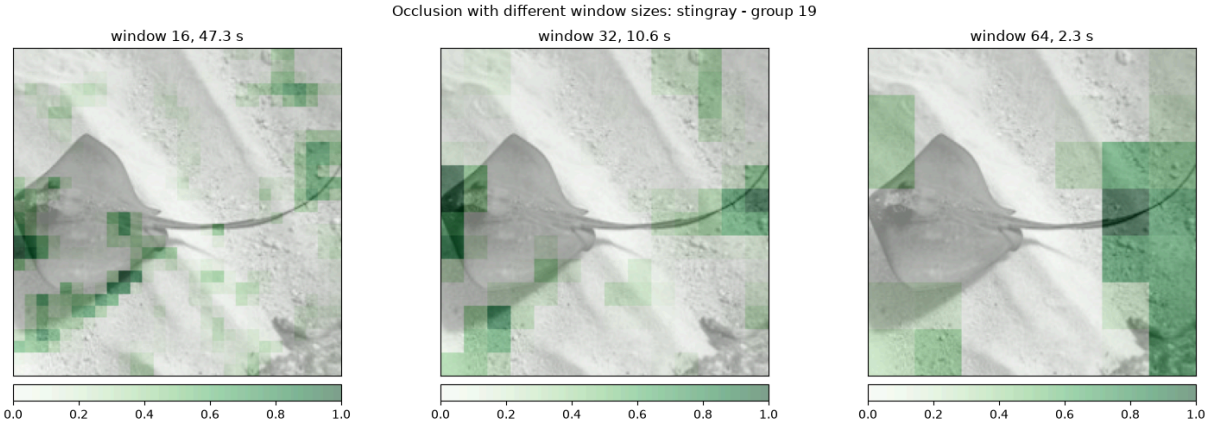
In [6]:
```
# (a) the dog and ram image: occlusion for the best dog class as well (which part
of the image is the dog?)
rgb_dr, x_dr = load_image("images/dog_and_ram.jpg")
with torch.no_grad():
    prob_dr = model(x_dr).softmax(1)[0]
dog_idx = [i for i in prob_dr.argsort(descending=True).tolist() if 151 <= i <=
268][0]   # ImageNet dog classes
```

```
print(f"best dog class: {labels[dog_idx]} (P = {prob_dr[dog_idx]:.3f})")
_ = show_occlusion("images/dog_and_ram.jpg", idx=dog_idx)
```

```
best dog class: Welsh springer spaniel (P = 0.034)
images/dog_and_ram.jpg
ram                              0.047
Welsh springer spaniel           0.034
Border collie                    0.027
llama                            0.023
Brittany spaniel                 0.020
```



In [7]:
```
# (c) size of the occlusion window: 16, 32 and 64 pixels (stride = half the
window)
rgb_o, x_o = load_image(OUR_IMAGE)
idx_o = labels.index("stingray")
window_rows = []
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, w in zip(axes, [16, 32, 64]):
    start = time.perf_counter()
    attr_w = occlusion_map(x_o, idx_o, window=w, stride=w // 2)
    seconds = time.perf_counter() - start
    n_positions = ((224 - w) // (w // 2) + 1) ** 2
    window_rows.append({"window": w, "stride": w // 2, "forward passes":
n_positions, "time (s)": round(seconds, 1),
                        "largest drop in P": attr_w.max().round(3)})
    viz.visualize_image_attr(attr_w, rgb_o, method="blended_heat_map",
sign="positive", show_colorbar=True,
                             plt_fig_axis=(fig, ax), use_pyplot=False,
title=f"window {w}, {seconds:.1f} s")
fig.suptitle(f"Occlusion with different window sizes: stingray - group
{GROUP_NUMBER}")
plt.tight_layout()
plt.show()
pd.DataFrame(window_rows).set_index("window")
```



Out[7]:

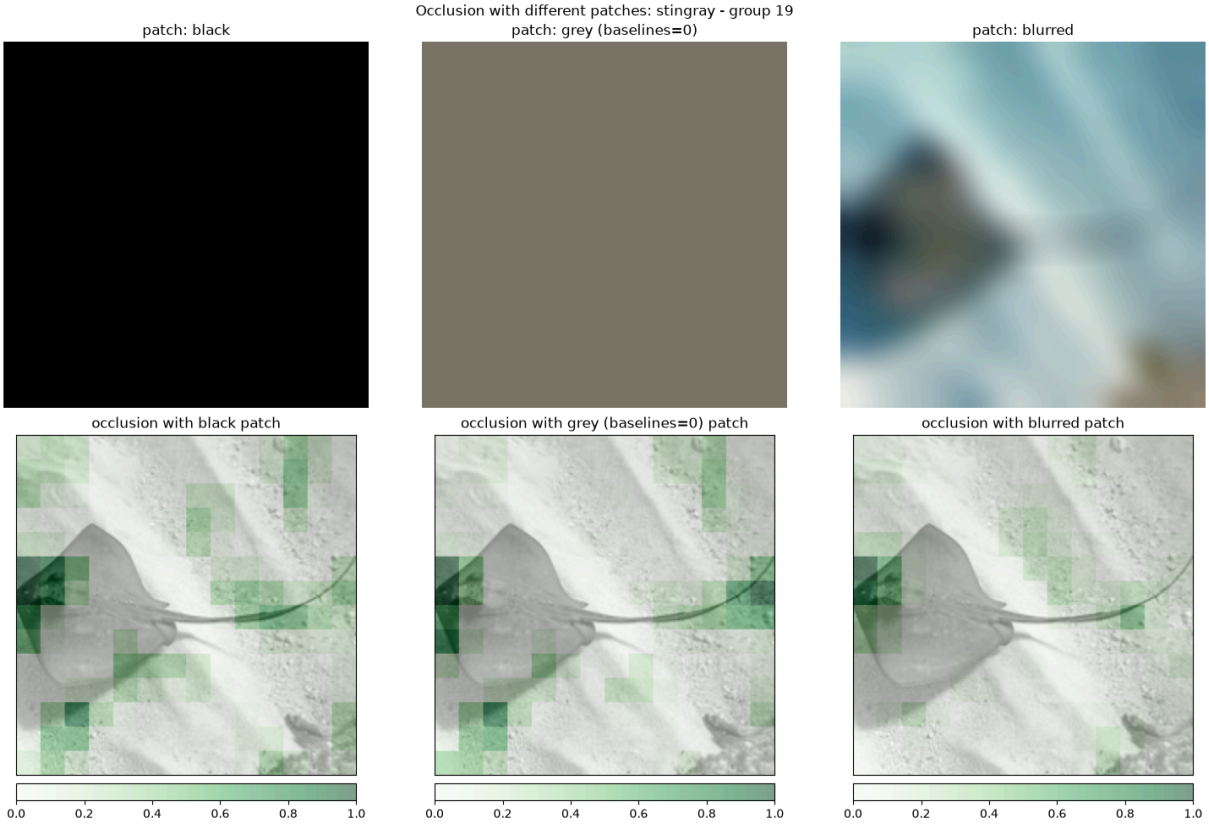
window	stride	forward passes	time (s)	largest drop in P
16	8	729	47.3	0.178
32	16	169	10.6	0.236
64	32	36	2.3	0.238

In [8]:

```
# (d) the patch that replaces the window: black, grey and blurred.
# The model input is normalised, so baselines=0 is the average ImageNet colour
(grey), not black.
mean = torch.tensor(preprocess.mean).view(1, 3, 1, 1)
std = torch.tensor(preprocess.std).view(1, 3, 1, 1)
patches = {"black": ((0 - mean) / std).expand_as(x_o),
           "grey (baselines=0)": torch.zeros_like(x_o),
           "blurred": T.GaussianBlur(kernel_size=31, sigma=10)(x_o)}

patch_maps = {}
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
for col, (name, base) in enumerate(patches.items()):
    shown = (base * std + mean).clamp(0, 1)[0].permute(1, 2, 0).numpy()
    axes[0, col].imshow(shown)
    axes[0, col].set_title(f"patch: {name}")
    axes[0, col].axis("off")
    patch_maps[name] = occlusion_map(x_o, idx_o, baselines=base)
    viz.visualize_image_attr(patch_maps[name], rgb_o, method="blended_heat_map",
sign="positive",
                             show_colorbar=True, plt_fig_axis=(fig, axes[1, col]),
use_pyplot=False,
                             title=f"occlusion with {name} patch")
fig.suptitle(f"Occlusion with different patches: stingray - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()

flat = {k: v.mean(axis=2)[::16, ::16].ravel() for k, v in patch_maps.items()}
print("largest drop in P(stingray):", {k: round(float(v.max()), 3) for k, v in
patch_maps.items()})
print("share of windows that raise P(stingray):", {k: round(float((v < 0).mean()),
3) for k, v in flat.items()})
print("Spearman correlation between the maps:")
pd.DataFrame({a_: {b_: spearmanr(flat[a_], flat[b_])[0] for b_ in flat} for a_ in
flat}).round(3)
```




```
largest drop in P(stingray): {'black': 0.245, 'grey (baselines=0)': 0.236,
'blurred': 0.46}
share of windows that raise P(stingray): {'black': 0.48, 'grey (baselines=0)':
0.536, 'blurred': 0.485}
Spearman correlation between the maps:
```

Out[8]:

	black	grey (baselines=0)	blurred
black	1.000	0.807	0.687
grey (baselines=0)	0.807	1.000	0.533
blurred	0.687	0.533	1.000

Your answers:

a) Occlusion slides a grey 32 × 32 patch over the image (stride 16) and records how much the probability of the class drops. Green means that covering this part lowers the probability, so the part helps the class. Red means that covering it *raises* the probability.

- **Basketball** (running example, P = 0.245): the important part is in the middle of the ball, where the black seams and the printed text are. The edge of the ball and the background barely matter, so the model recognises a basketball by its seams and surface pattern, not by its round shape.
- **Stingray** (our image): the model is not sure. P(stingray) = 0.251, and *electric ray* is close behind with 0.203. The green parts are the front edge and head of the ray (left side), some spots along the outline and the tail, but also patches of sand in the corners. Covering the middle of the body makes P(stingray) go *up* (red). Our reading is that the body is what makes the animal look like a ray in general, and also like an electric ray. Once the body is covered, the competing class loses more than stingray does, and the edges and the long thin tail decide. So the model does use parts of the animal, but it also uses sand. It knows "flat fish on sand" more than "stingray".

- **Dog and ram**: the top class is *ram* with only P = 0.047, then *Welsh springer spaniel* (0.034) and *Border collie* (0.027). For *ram* the green parts are the ram's face, its horn and its legs, and covering the dog raises P(ram). For the spaniel it is the other way around: the dog's face helps and parts of the ram count against it. So the model separates the two animals well and looks at the right places. The low probabilities show that it does not "know" that one image can hold two objects: the softmax divides the probability between them.

b) A model learns whatever separates the classes in the training data, including things that are only there by accident. In the lecture example a husky was classified as a wolf because of the snow in the background: in the training data wolves were almost always on snow. Our stingray shows the same risk, since the sand around the animal helps the prediction. So that the model learns what humans use (shape, eyes, tail, texture of the animal), the data set needs:

- the object in many different backgrounds, poses, sizes, lighting conditions and with partial occlusion, so that the background no longer predicts the class
- no systematic differences between classes other than the object itself, such as one class always photographed on sand or always with a watermark
- enough examples of classes that look alike (stingray / electric ray, dog breeds), so the model must learn the small details that separate them
- correct labels

Explanation methods like occlusion are a way to check this afterwards.

c) Window 16 (stride 8) needs 729 forward passes and took 47.3 s. Window 32 (stride 16): 169 passes, 10.6 s. Window 64 (stride 32): 36 passes, 2.3 s. The run time scales with the number of window positions, about $(224 / \text{stride})^2$. With window 16 the map is detailed but noisy: one small patch rarely hides a whole part of the ray, so the largest drop in P is only 0.178, and many single patches on the sand light up. With window 64 the map is coarse (blocks of 64 pixels) and mixes object and background: the biggest drop (0.238) is now on the sand and tail on the right, and the head of the ray falls apart over several blocks. Window 32 is a compromise. The window should have about the size of the parts we want to find.

d) Black patch: largest drop 0.245. Grey patch (the default `baselines=0`, which after normalisation is the average ImageNet colour, not black): 0.236. Blurred patch: 0.460. The maps agree only partly: the Spearman correlation between them is 0.81 (black–grey), 0.69 (black–blurred) and 0.53 (grey–blurred). With black and grey the corners of the sand also light up. With the blurred patch mostly the head of the ray remains.

The patch matters because occlusion measures "prediction with this part" minus "prediction with the patch instead". A black or grey square adds a new object with sharp edges that never occurs in natural images, so part of the drop comes from the strange square and not from losing the information. A blurred patch keeps the colours and only removes detail, so it is closer to "remove the information". There is no neutral patch, and the explanation always depends on what we compare with.

Q2) Unsupervised explainers

Note: this question does not require a deep learning model.

a) **[Basic]** Using PCA, find the number of components that each explain at least 10% of the variance (explained variance ratio ≥ 0.1). How many components do you need to explain 90% of the variance?

b) **[Basic]** Perform PCA and t-SNE on the Fashion-MNIST data set using the code below.

c) **[Basic]** Look at your two assigned classes. Find test images that are classified as the wrong one of the two, and mark them in the t-SNE plot. Explain what can be interpreted using an unsupervised method like this in explaining an incorrect classification.

d) **[Regular]** Run t-SNE with perplexity 5, 30 and 100. What can and what can you not conclude from a t-SNE plot?

e) **[Advanced]** Show the first four PCA components as 28×28 images. What does each component capture?

> **Hint:** `pca.explained_variance_ratio_`, `np.cumsum(...)`; `TSNE(n_components=2, perplexity=30, init="pca", random_state=S)`; misclassified: `(y_te == c1) & (pred == c2)`

In [9]:
```
train = datasets.FashionMNIST("data", train=True, download=True)
test = datasets.FashionMNIST("data", train=False, download=True)

rng = np.random.default_rng(S)
i_train = rng.choice(len(train), 5000, replace=False)
i_test = rng.choice(len(test), 3000, replace=False)
X_tr = train.data.numpy()[i_train].reshape(-1, 784) / 255
y_tr = train.targets.numpy()[i_train]
X_te = test.data.numpy()[i_test].reshape(-1, 784) / 255
y_te = test.targets.numpy()[i_test]
```

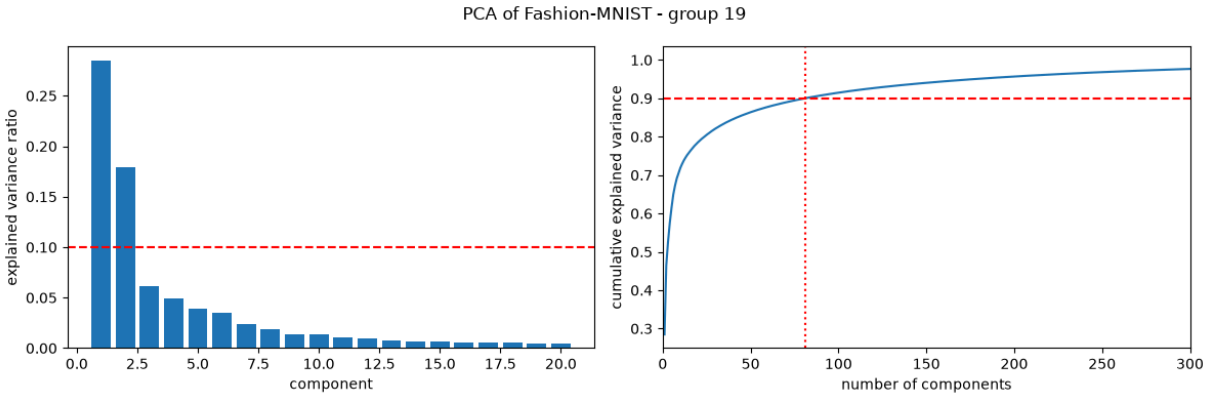
In [10]:
```
pca = PCA().fit(X_tr)
# (a) explained variance ratio of each component and the cumulative sum
ratio = pca.explained_variance_ratio_
cum = np.cumsum(ratio)
print("components with explained variance ratio >= 0.1:", np.where(ratio >= 0.1)
[0] + 1, ratio[ratio >= 0.1].round(3))
for target in [0.5, 0.8, 0.9, 0.95]:
    print(f"components needed for {target:.0%} of the variance: {np.argmax(cum >=
target) + 1}")

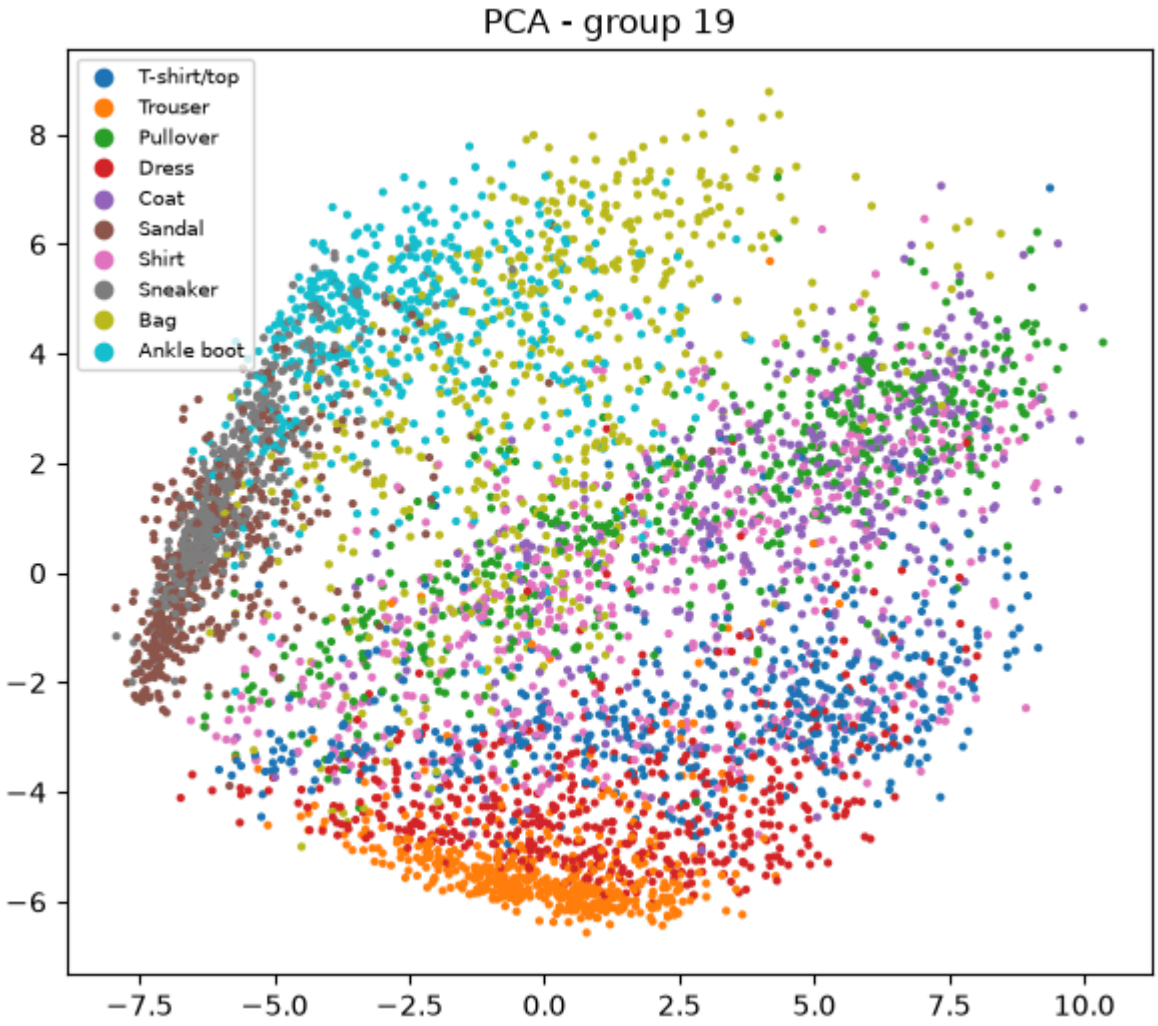
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(range(1, 21), ratio[:20])
axes[0].axhline(0.1, color="red", ls="--")
axes[0].set_xlabel("component")
axes[0].set_ylabel("explained variance ratio")
axes[1].plot(range(1, len(cum) + 1), cum)
axes[1].axhline(0.9, color="red", ls="--")
axes[1].axvline(np.argmax(cum >= 0.9) + 1, color="red", ls=":")
axes[1].set_xlim(0, 300)
```

```
axes[1].set_xlabel("number of components")
axes[1].set_ylabel("cumulative explained variance")
fig.suptitle(f"PCA of Fashion-MNIST - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()

Z = PCA(n_components=2).fit_transform(X_tr)
plt.figure(figsize=(7, 6))
sc = plt.scatter(Z[:, 0], Z[:, 1], c=y_tr, cmap="tab10", s=4)
plt.legend(sc.legend_elements()[0], FASHION_CLASSES, fontsize=7)
plt.title(f"PCA - group {GROUP_NUMBER}")
plt.show()
```

```
components with explained variance ratio >= 0.1: [1 2] [0.285 0.18 ]
components needed for 50% of the variance: 3
components needed for 80% of the variance: 24
components needed for 90% of the variance: 81
components needed for 95% of the variance: 177
```





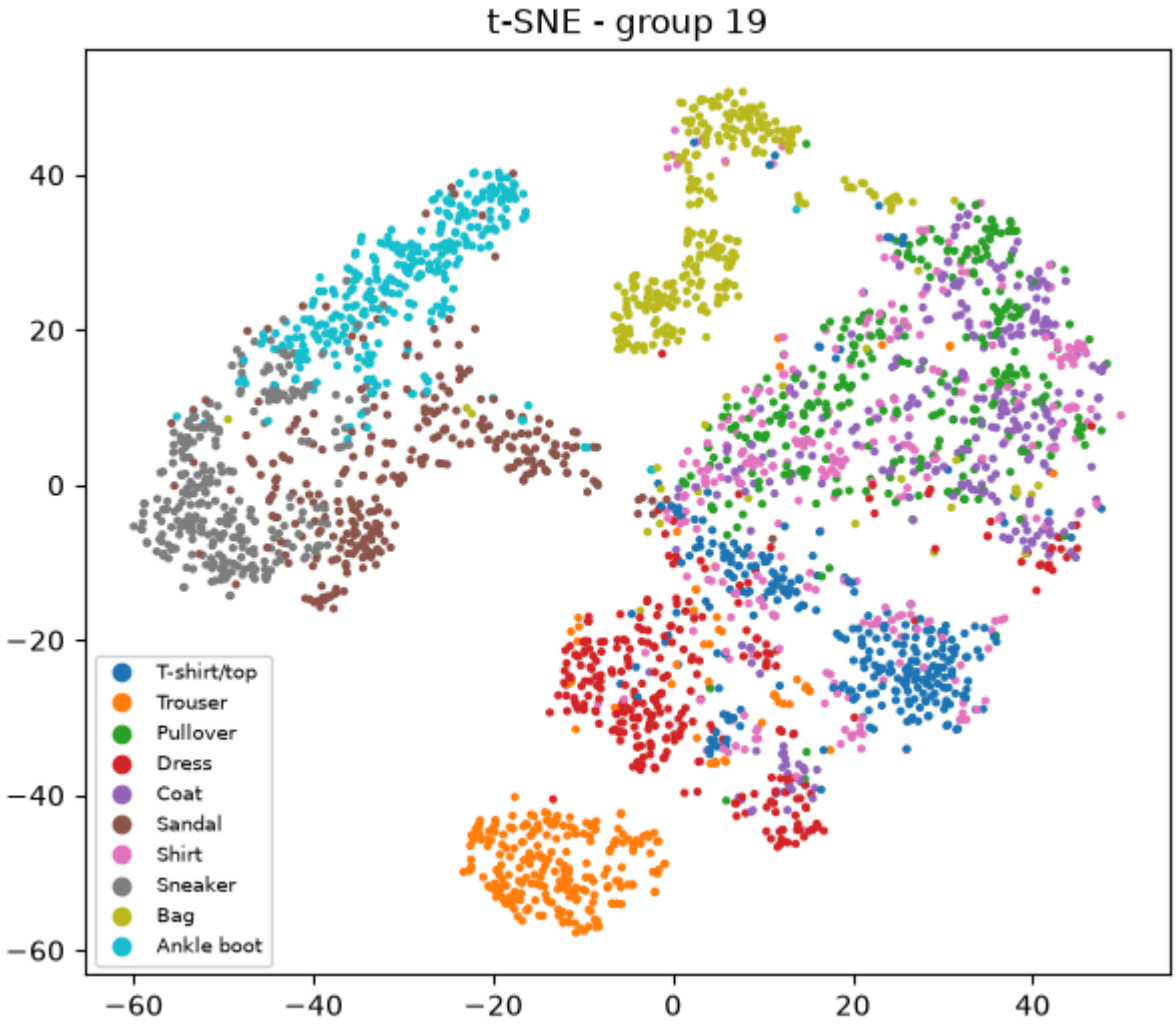
In [11]:
```
# t-SNE of the test images (takes about a minute)
E = TSNE(n_components=2, perplexity=30, init="pca",
random_state=S).fit_transform(X_te)

# A simple classifier, to find misclassified images
clf = LogisticRegression(max_iter=300).fit(X_tr, y_tr)
pred = clf.predict(X_te)
print("test accuracy:", (pred == y_te).mean())

plt.figure(figsize=(7, 6))
sc = plt.scatter(E[:, 0], E[:, 1], c=y_te, cmap="tab10", s=4)
plt.legend(sc.legend_elements()[0], FASHION_CLASSES, fontsize=7)
plt.title(f"t-SNE - group {GROUP_NUMBER}")
plt.show()

c1, c2 = a["fashion_pair"]
print("Your classes:", a["fashion_pair_names"])
```

```
test accuracy: 0.82
```




```
Your classes: ('Coat', 'Shirt')
```

In [12]:

```
# (c) test images of c1 predicted as c2, and the other way around, marked in the
t-SNE plot
wrong_12 = (y_te == c1) & (pred == c2)
wrong_21 = (y_te == c2) & (pred == c1)
print(f"{FASHION_CLASSES[c1]} predicted as {FASHION_CLASSES[c2]}: {wrong_12.sum()}
of {(y_te == c1).sum()}")
print(f"{FASHION_CLASSES[c2]} predicted as {FASHION_CLASSES[c1]}: {wrong_21.sum()}
of {(y_te == c2).sum()}")
print("confusion of the two classes (rows: true, columns: predicted):")
display(pd.crosstab(pd.Series(y_te,
name="true").map(dict(enumerate(FASHION_CLASSES))),
                    pd.Series(pred,
name="predicted").map(dict(enumerate(FASHION_CLASSES))))
        .loc[[FASHION_CLASSES[c1], FASHION_CLASSES[c2]]])

fig, axes = plt.subplots(1, 2, figsize=(16, 7))
for ax, zoom in zip(axes, [False, True]):
    ax.scatter(E[:, 0], E[:, 1], c="lightgrey", s=3)
    for c, color in [(c1, "tab:blue"), (c2, "tab:orange")]:
        m = y_te == c
        ax.scatter(E[m, 0], E[m, 1], color=color, s=6, label=f"
{FASHION_CLASSES[c]} (true)")
    ax.scatter(E[wrong_12, 0], E[wrong_12, 1], marker="x", color="darkblue", s=40,
               label=f"{FASHION_CLASSES[c1]} predicted as {FASHION_CLASSES[c2]}")
    ax.scatter(E[wrong_21, 0], E[wrong_21, 1], marker="+", color="red", s=60,
               label=f"{FASHION_CLASSES[c2]} predicted as {FASHION_CLASSES[c1]}")
    if zoom:
```

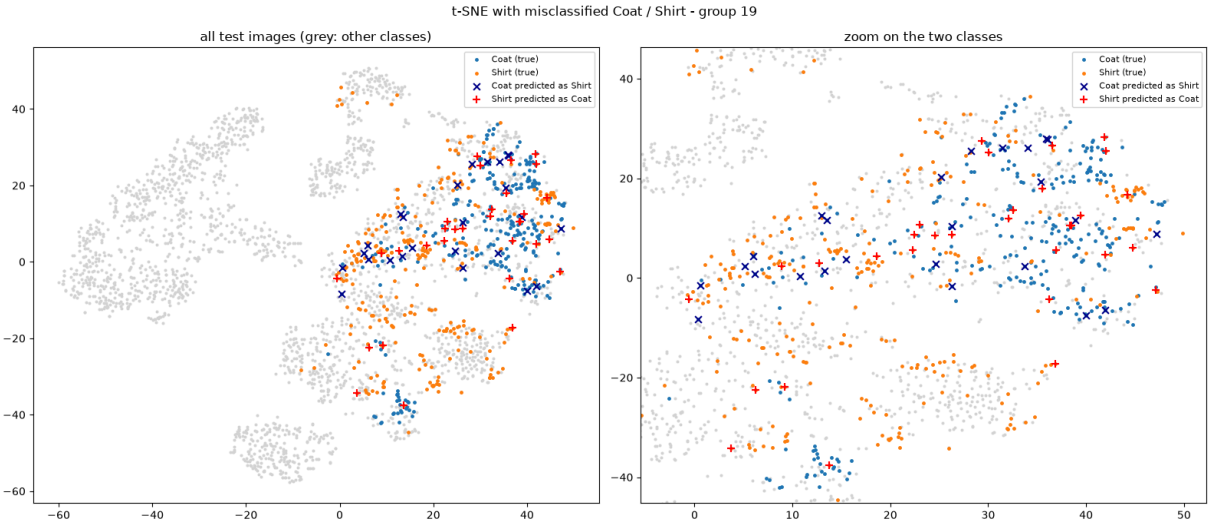
```
        m = (y_te == c1) | (y_te == c2)
        ax.set_xlim(np.percentile(E[m, 0], [1, 99]) + [-5, 5])
        ax.set_ylim(np.percentile(E[m, 1], [1, 99]) + [-5, 5])
        ax.set_title("zoom on the two classes")
    else:
        ax.set_title("all test images (grey: other classes)")
    ax.legend(fontsize=8)
fig.suptitle(f"t-SNE with misclassified {FASHION_CLASSES[c1]} /
{FASHION_CLASSES[c2]} - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()

# where do the misclassified images lie? share of their 10 nearest t-SNE
neighbours of each class
from sklearn.neighbors import NearestNeighbors
nn_tsne = NearestNeighbors(n_neighbors=11).fit(E)
def neighbour_share(mask):
    nbr = nn_tsne.kneighbors(E[mask], return_distance=False)[:, 1:]
    return {FASHION_CLASSES[c]: round((y_te[nbr] == c).mean(), 3) for c in [c1,
c2]}
print("10 nearest t-SNE neighbours of the misclassified images:")
print(f"  {FASHION_CLASSES[c1]} predicted as {FASHION_CLASSES[c2]}:",
neighbour_share(wrong_12))
print(f"  {FASHION_CLASSES[c2]} predicted as {FASHION_CLASSES[c1]}:",
neighbour_share(wrong_21))
print(f"  correctly classified {FASHION_CLASSES[c1]}:", neighbour_share((y_te ==
c1) & (pred == c1)))
print(f"  correctly classified {FASHION_CLASSES[c2]}:", neighbour_share((y_te ==
c2) & (pred == c2)))

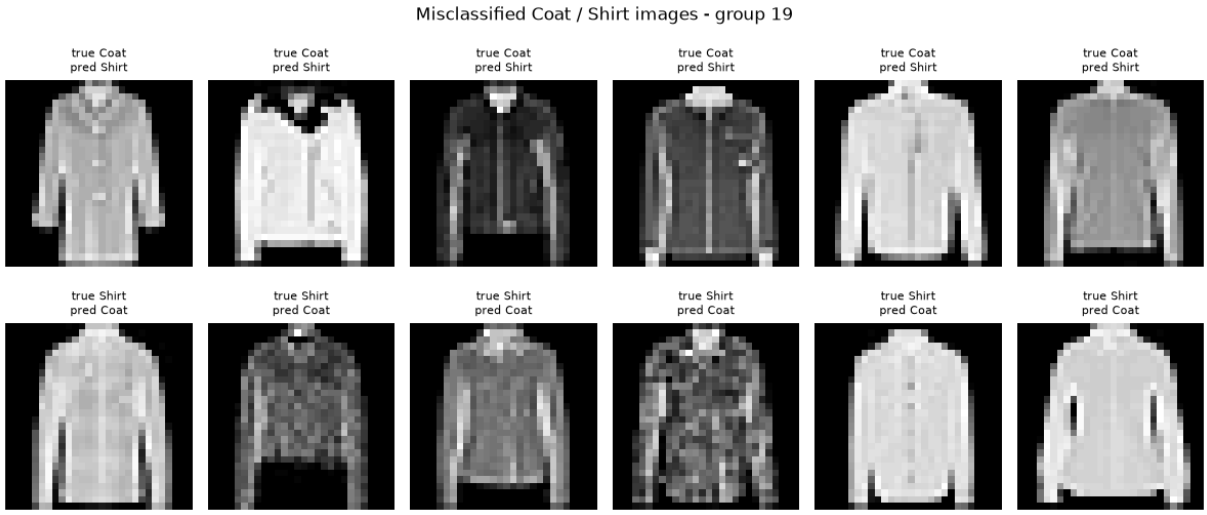
# some of the misclassified images
show = np.concatenate([np.where(wrong_12)[0][:6], np.where(wrong_21)[0][:6]])
fig, axes = plt.subplots(2, 6, figsize=(12, 5.5))
for ax, i in zip(axes.ravel(), show):
    ax.imshow(X_te[i].reshape(28, 28), cmap="gray")
    ax.set_title(f"true {FASHION_CLASSES[y_te[i]]}\npred
{FASHION_CLASSES[pred[i]]}", fontsize=8)
    ax.axis("off")
fig.suptitle(f"Misclassified {FASHION_CLASSES[c1]} / {FASHION_CLASSES[c2]} images
- group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()
```

```
Coat predicted as Shirt: 25 of 297
Shirt predicted as Coat: 31 of 317
confusion of the two classes (rows: true, columns: predicted):
```

predicted	Ankle boot	Bag	Coat	Dress	Pullover	Sandal	Shirt	Sneaker	T-shirt/top	Trouser
true										
Coat	0	1	223	13	32	0	25	0	2	1
Shirt	0	7	31	15	46	1	169	0	47	1

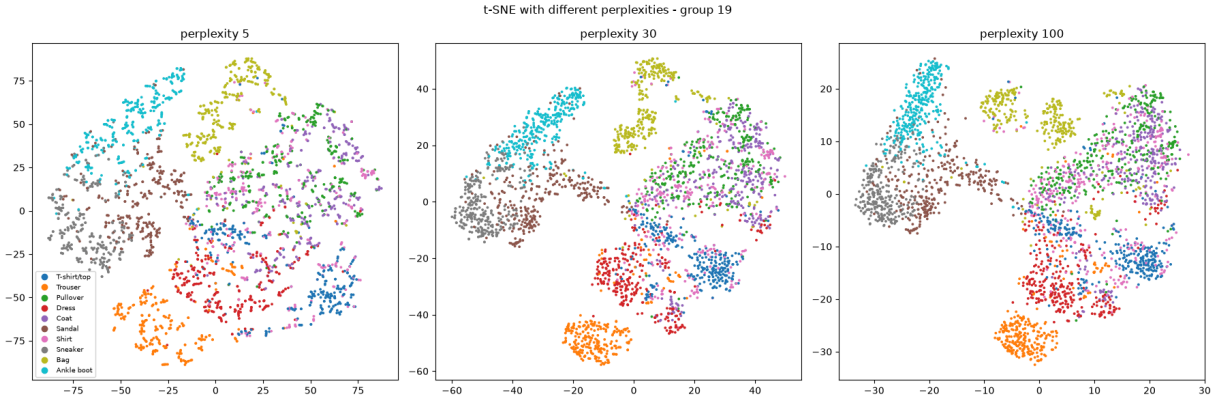



```
10 nearest t-SNE neighbours of the misclassified images:
  Coat predicted as Shirt: {'Coat': np.float64(0.312), 'Shirt': np.float64(0.276)}
  Shirt predicted as Coat: {'Coat': np.float64(0.439), 'Shirt': np.float64(0.187)}
  correctly classified Coat: {'Coat': np.float64(0.527), 'Shirt': np.float64(0.165)}
  correctly classified Shirt: {'Coat': np.float64(0.149), 'Shirt':
np.float64(0.483)}
```



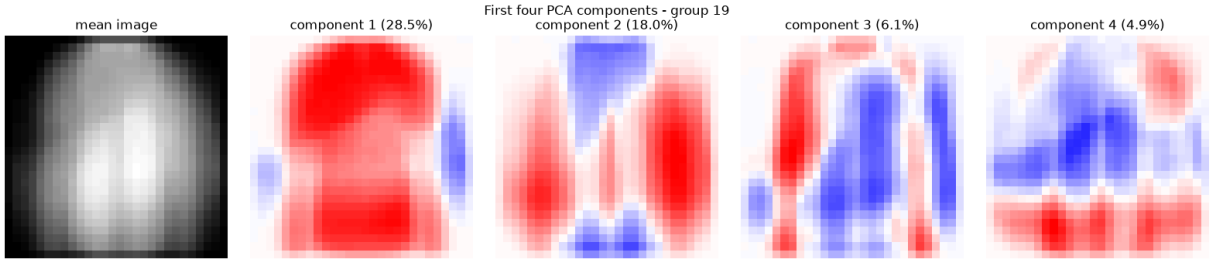
In [13]:

```python
# (d) t-SNE with perplexity 5, 30 and 100
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, perplexity in zip(axes, [5, 30, 100]):
    start = time.perf_counter()
    E_p = E if perplexity == 30 else TSNE(n_components=2, perplexity=perplexity,
init="pca",
                                          random_state=S).fit_transform(X_te)
    sc = ax.scatter(E_p[:, 0], E_p[:, 1], c=y_te, cmap="tab10", s=3)
    ax.set_title(f"perplexity {perplexity}")
axes[0].legend(sc.legend_elements()[0], FASHION_CLASSES, fontsize=7)
fig.suptitle(f"t-SNE with different perplexities - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()
```



In [14]:
```
# (e) the first four PCA components as 28 x 28 images (red = positive weight, blue
= negative weight)
fig, axes = plt.subplots(1, 5, figsize=(16, 3.5))
axes[0].imshow(pca.mean_.reshape(28, 28), cmap="gray")
axes[0].set_title("mean image")
for k, ax in enumerate(axes[1:]):
    comp = pca.components_[k].reshape(28, 28)
    lim = np.abs(comp).max()
    ax.imshow(comp, cmap="bwr", vmin=-lim, vmax=lim)
    ax.set_title(f"component {k + 1} ({ratio[k]:.1%})")
for ax in axes:
    ax.axis("off")
fig.suptitle(f"First four PCA components - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()

# which classes lie at the two ends of each component?
scores = pca.transform(X_tr)[:, :4]
ends = pd.DataFrame({f"component {k + 1}": pd.Series(scores[:,
k]).groupby(y_tr).mean().values
                     for k in range(4)}, index=FASHION_CLASSES)
ends.round(2)
```



Out[14]:

	component 1	component 2	component 3	component 4
T-shirt/top	2.82	-2.34	-0.16	0.91
Trouser	0.04	-5.33	-1.17	-1.24
Pullover	3.01	1.26	2.24	-0.02
Dress	1.19	-4.07	-1.43	-0.07
Coat	4.16	0.90	1.50	-0.52
Sandal	-5.92	0.43	0.72	0.64
Shirt	2.03	-0.27	1.49	0.53
Sneaker	-5.80	1.50	0.06	-1.99
Bag	0.07	3.34	0.13	0.24
Ankle boot	-2.37	4.27	-3.27	1.36

Your answers:

a) Only two components have an explained variance ratio of at least 0.1: component 1 (0.285) and component 2 (0.180). For 90% of the variance we need **81 components** (3 for 50%, 24 for 80%, 177 for 95%). Two components already hold almost half of the variance, but the rest is spread over many small components.

b) The PCA and t-SNE plots are above. In the PCA plot the footwear (sandal, sneaker, ankle boot) is on the left, trousers and dresses at the bottom, and the upper-body clothes (T-shirt, pullover, coat, shirt) overlap strongly on the right. t-SNE separates the classes much better. Trousers, bags and the three shoe classes form their own clusters, but pullover, coat and shirt still lie in one large mixed cloud.

c) The logistic regression (test accuracy 0.82) predicts 25 of the 297 coats as shirt and 31 of the 317 shirts as coat. Shirts are hard in general: 169 of 317 are correct, and many are predicted as T-shirt (47) or pullover (46).

In the t-SNE plot almost all misclassified coats and shirts lie in the region where the two classes are mixed, and not inside a clean cluster of one class. We checked this with the 10 nearest t-SNE neighbours. Of a correctly classified coat, 53% of the neighbours are coats and 17% shirts. Of a coat predicted as shirt, it is 31% coats and 28% shirts. A shirt predicted as coat even has more coats (44%) than shirts (19%) as neighbours. The example images show why: these are long-sleeved, buttoned, often dark or patterned garments, and at 28 × 28 pixels a coat and a shirt look almost the same.

What this tells us about an incorrect classification: the error is not a strange mistake of the model. The images are similar to images of the other class in pixel space, so the data itself does not separate them well. The method does not use the classifier, so it cannot say why the model chose one class or which pixels it used. It only shows that these images lie between the classes, and a better model or features beyond raw pixels are needed for them.

d) With perplexity 5 every point only looks at a few neighbours. The plot breaks into many small groups and loose points, and the large classes are cut into pieces. With perplexity 30 we get compact clusters per class. With perplexity 100 the global structure is clearer (footwear on one side, upper-body clothes on the other, trousers at the bottom), but the clusters are less sharp and the mixed region of shirt, coat and pullover gets bigger.

What we *can* conclude: which images are neighbours, so which classes are well separated in pixel space and which overlap. This holds in all three plots (trousers, bags and footwear separate, shirt/coat/pullover mixed). What we *can't* conclude: the distances between clusters, the size of a cluster, the density, and the position or axes of the plot. These depend on the perplexity and the random start, and t-SNE stretches and shrinks distances on purpose. A small gap between two clusters does not mean the classes are similar, and t-SNE also does not tell us anything about the classifier.

e) The mean image is a blurry piece of clothing. Each component adds (red) or removes (blue) brightness compared with it:

- **Component 1 (28.5%)**: almost all red over the whole middle, so it measures how much of the image is filled: a lot of bright fabric (coats 4.16, pullovers 3.01 on average) against little (sandals −5.92, sneakers −5.80).

- **Component 2 (18.0%)**: red at the left and right bottom, blue at the top centre and bottom middle. Shoes and bags (ankle boot 4.27, bag 3.34) score high, trousers (−5.33) and dresses (−4.07) low. It separates wide, low objects from tall, narrow ones with a gap between the legs.
- **Component 3 (6.1%)**: a red vertical band on the left against blue in the middle and on the right. It separates garments with long sleeves next to the body (pullover 2.24, coat 1.50, shirt 1.49) from ankle boots (−3.27) and dresses.
- **Component 4 (4.9%)**: red at the bottom, blue in the upper middle. It mostly contrasts the lower part of the image, for example the sole of shoes and the hem of clothes, against the upper part (ankle boot 1.36 against sneaker −1.99).

PCA thus finds global shape (size, width, sleeves). These are the same differences that make coat and shirt hard to separate, since they score almost the same on all four components.

Q3) Gradient based explainers

a) **[Basic]** Generate a Grad-CAM heatmap overlay for your assigned image. A running example of an aircraft carrier and a submarine is given.

b) **[Basic]** Explain how, when there are multiple objects in a frame, this could be used to identify each object individually. Show this with Grad-CAM for the top 3 classes of the aircraft carrier and submarine image (or another image with several objects).

c) **[Regular]** Sanity check: make a copy of the model and randomise the weights of the last layer (`fc`), then also of `layer4` . Compute a saliency map and Grad-CAM for your assigned image again. Do the maps change? What does it mean if a map does *not* change?

d) **[Advanced]** Show a saliency map, Integrated Gradients and Grad-CAM side by side for your assigned image. Which one is easiest to understand? Which one shows most detail?

e) **[Advanced]** Remove the most important parts of your image step by step (according to each method) and plot the class probability. Which method finds the parts that matter most for the model?

> **Hint:** `cam = GradCAM(model=model, target_layers=[model.layer4[-1]])` ; `heat = cam(input_tensor=x, targets=[ClassifierOutputTarget(idx)])[0]` ; `Saliency(model).attribute(x, target=idx)` , `captum.attr.IntegratedGradients` ; random weights: `random_model = copy.deepcopy(model)` ; `random_model.fc.reset_parameters()`

In [15]:
```
# running example (the file in images/ is called
submarine_and_aircraft_carrier.jpg)
EXAMPLE_GRADCAM_IMAGE = "images/submarine_and_aircraft_carrier.jpg"

rgb, x = load_image(EXAMPLE_GRADCAM_IMAGE)
idx = top_k(x)[0]

cam = GradCAM(model=model, target_layers=[model.layer4[-1]])
```

```
heat = cam(input_tensor=x, targets=[ClassifierOutputTarget(idx)])[0]

plt.imshow(show_cam_on_image(rgb, heat, use_rgb=True))
plt.title(f"Grad-CAM: {labels[idx]} - group {GROUP_NUMBER}")
plt.axis("off")
plt.show()
```

```
aircraft carrier              0.108
submarine                     0.084
speedboat                     0.006
amphibian                     0.004
warplane                      0.004
```



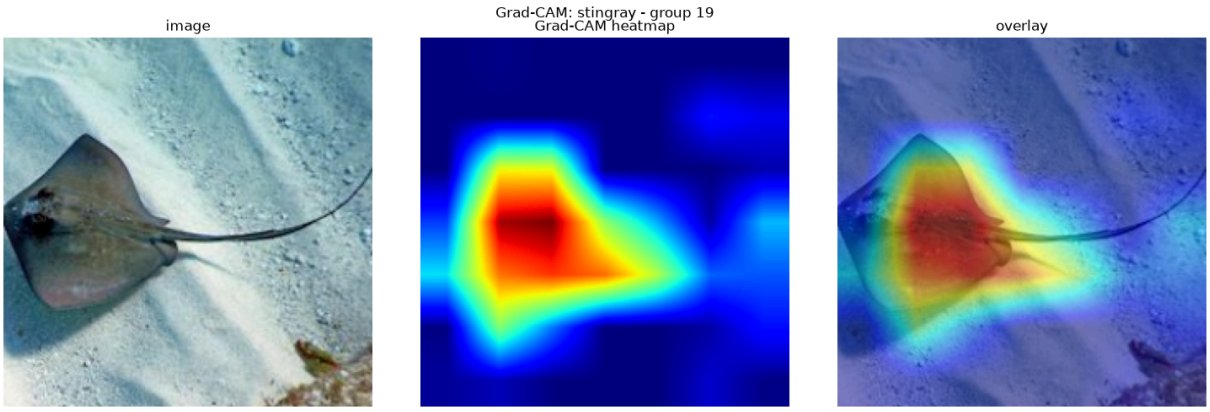
In [16]:

```
# (a) Grad-CAM for our assigned image
def gradcam(m, x, idx):
    cam_m = GradCAM(model=m, target_layers=[m.layer4[-1]])
    return cam_m(input_tensor=x, targets=[ClassifierOutputTarget(idx)])[0]

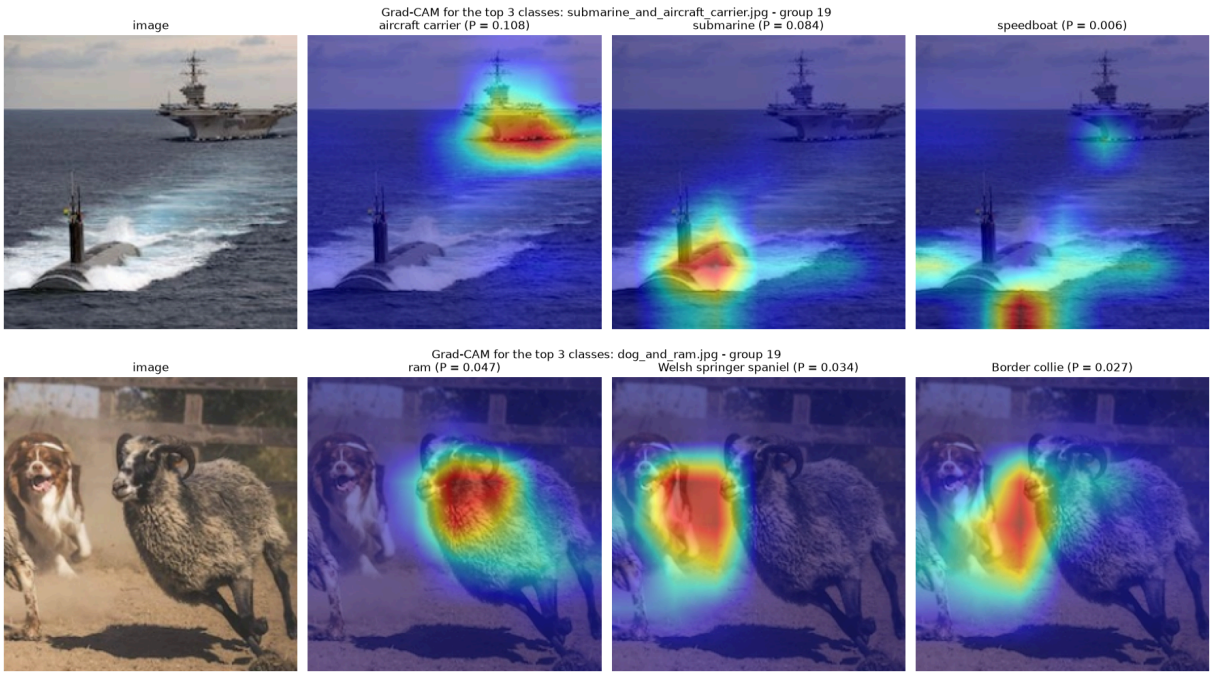

rgb_o, x_o = load_image(OUR_IMAGE)
top_o = top_k(x_o)
heat_o = gradcam(model, x_o, idx_o)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(rgb_o)
axes[0].set_title("image")
axes[1].imshow(heat_o, cmap="jet")
axes[1].set_title("Grad-CAM heatmap")
axes[2].imshow(show_cam_on_image(rgb_o, heat_o, use_rgb=True))
axes[2].set_title("overlay")
for ax in axes:
    ax.axis("off")
fig.suptitle(f"Grad-CAM: {labels[idx_o]} - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()
```

```
stingray                0.251
electric ray            0.203
harvestman              0.002
shovel                  0.002
cockroach               0.002
```



In [17]:
```python
# (b) Grad-CAM for the top 3 classes of the images with several objects
for path in [EXAMPLE_GRADCAM_IMAGE, "images/dog_and_ram.jpg"]:
    rgb_m, x_m = load_image(path)
    with torch.no_grad():
        prob_m = model(x_m).softmax(1)[0]
    top3 = prob_m.topk(3).indices.tolist()
    fig, axes = plt.subplots(1, 4, figsize=(18, 5))
    axes[0].imshow(rgb_m)
    axes[0].set_title("image")
    for ax, i in zip(axes[1:], top3):
        ax.imshow(show_cam_on_image(rgb_m, gradcam(model, x_m, i), use_rgb=True))
        ax.set_title(f"{labels[i]} (P = {prob_m[i]:.3f})")
    for ax in axes:
        ax.axis("off")
    fig.suptitle(f"Grad-CAM for the top 3 classes: {path.split('/')[-1]} - group
{GROUP_NUMBER}")
    plt.tight_layout()
    plt.show()
```



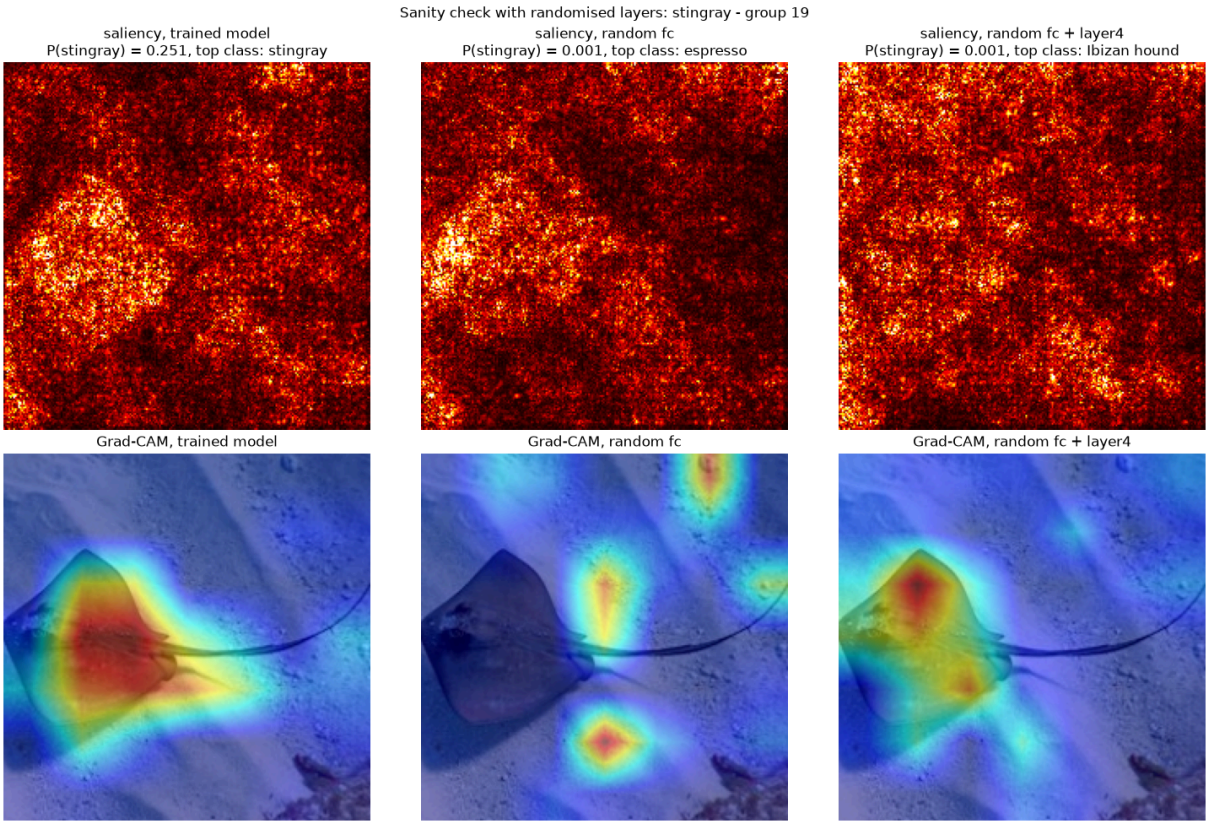
In [18]:
```python
# (c) Sanity check: a copy of the model with random weights in the last layer,
then also in layer4
```

```
torch.manual_seed(S)
random_fc = copy.deepcopy(model)
random_fc.fc.reset_parameters()
random_l4 = copy.deepcopy(random_fc)
for m in random_l4.layer4.modules():
    if hasattr(m, "reset_parameters"):
        m.reset_parameters()


def saliency_map(m, x, idx):
    return Saliency(m).attribute(x, target=idx).abs().max(dim=1)[0]
[0].detach().numpy()


variants = {"trained model": model, "random fc": random_fc, "random fc + layer4":
random_l4}
maps = {}
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
for col, (name, m) in enumerate(variants.items()):
    with torch.no_grad():
        p_m = m(x_o).softmax(1)[0]
    maps[(name, "saliency")] = saliency_map(m, x_o, idx_o)
    maps[(name, "Grad-CAM")] = gradcam(m, x_o, idx_o)
    axes[0, col].imshow(maps[(name, "saliency")], cmap="hot",
                        vmax=np.percentile(maps[(name, "saliency")], 99.5))
    axes[0, col].set_title(f"saliency, {name}\nP(stingray) = {p_m[idx_o]:.3f}, top
class: {labels[p_m.argmax()]}")
    axes[1, col].imshow(show_cam_on_image(rgb_o, maps[(name, "Grad-CAM")],
use_rgb=True))
    axes[1, col].set_title(f"Grad-CAM, {name}")
for ax in axes.ravel():
    ax.axis("off")
fig.suptitle(f"Sanity check with randomised layers: stingray - group
{GROUP_NUMBER}")
plt.tight_layout()
plt.show()

# how similar is each map to the map of the trained model? (Spearman correlation
of the pixel values)
pd.DataFrame({method: {name: spearmanr(maps[("trained model", method)].ravel(),
maps[(name, method)].ravel())[0]
                       for name in variants}
              for method in ["saliency", "Grad-CAM"]}).round(3)
```



Out[18]:

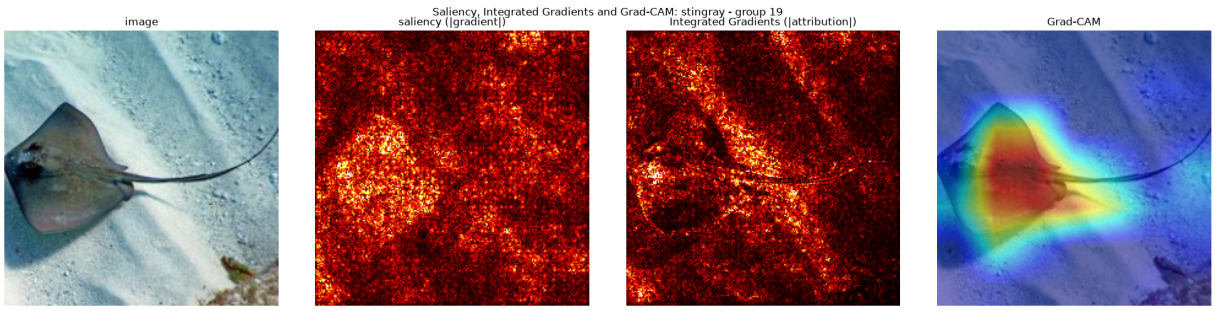
	saliency	Grad-CAM
trained model	1.000	1.000
random fc	0.330	-0.273
random fc + layer4	0.257	0.319

In [19]:

```
# (d) saliency, Integrated Gradients and Grad-CAM side by side for our image
sal_o = saliency_map(model, x_o, idx_o)
ig_attr = IntegratedGradients(model).attribute(x_o, target=idx_o,
baselines=torch.zeros_like(x_o),
                                               n_steps=50, internal_batch_size=10)
ig_o = ig_attr.abs().sum(dim=1)[0].detach().numpy()

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(rgb_o)
axes[0].set_title("image")
axes[1].imshow(sal_o, cmap="hot", vmax=np.percentile(sal_o, 99.5))
axes[1].set_title("saliency (|gradient|)")
axes[2].imshow(ig_o, cmap="hot", vmax=np.percentile(ig_o, 99.5))
axes[2].set_title("Integrated Gradients (|attribution|)")
axes[3].imshow(show_cam_on_image(rgb_o, heat_o, use_rgb=True))
axes[3].set_title("Grad-CAM")
for ax in axes:
    ax.axis("off")
fig.suptitle(f"Saliency, Integrated Gradients and Grad-CAM: stingray - group
{GROUP_NUMBER}")
plt.tight_layout()
plt.show()

with torch.no_grad():
    logit_x, logit_0 = model(x_o)[0, idx_o].item(), model(torch.zeros_like(x_o))
[0, idx_o].item()
print(f"IG completeness: sum of attributions {ig_attr.sum().item():.3f}, "
      f"logit(image) - logit(baseline) = {logit_x - logit_0:.3f}")
```



IG completeness: sum of attributions 4.372, logit(image) - logit(baseline) = 6.258

In [20]:

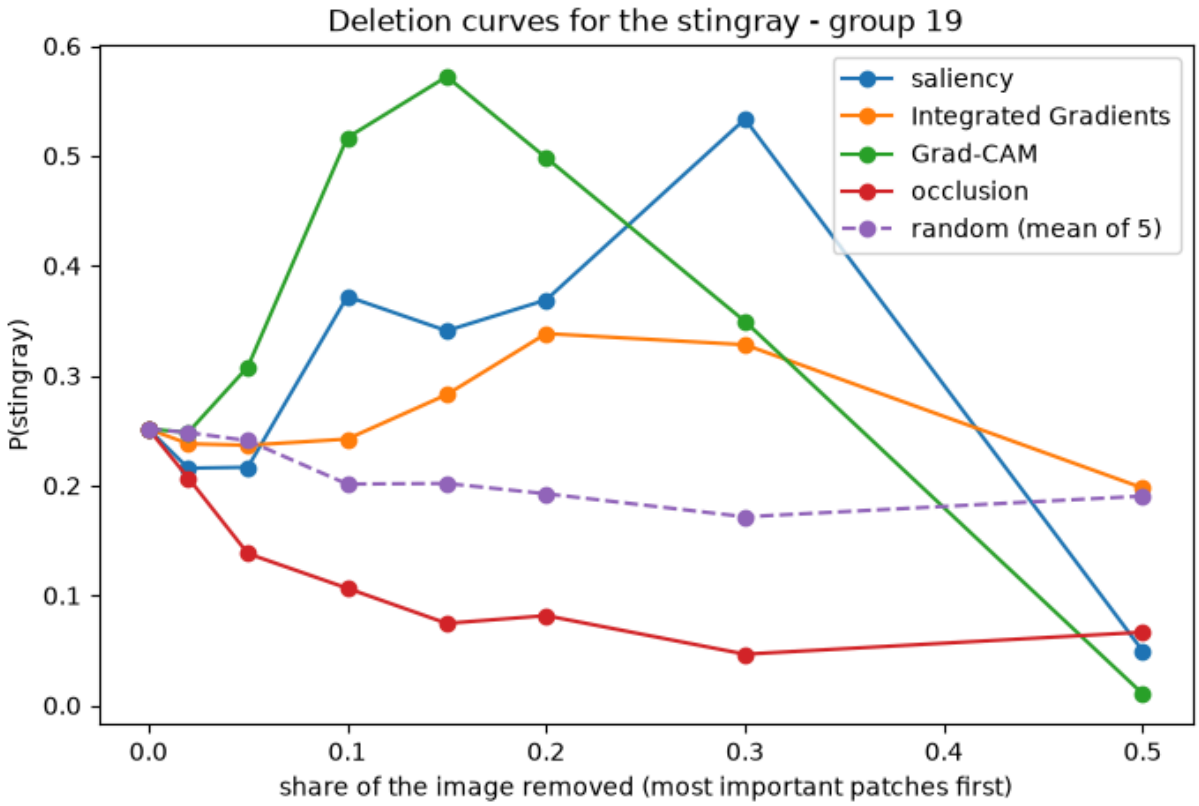
```
# (e) remove the most important parts step by step and follow P(stingray).
# The image is cut in 14 x 14 patches of 16 x 16 pixels. Each method ranks the
patches by its average
# importance, and we replace the patches with the average colour (0 after
normalisation), most important first.
rng_px = np.random.default_rng(S)
importance = {"saliency": sal_o, "Integrated Gradients": ig_o, "Grad-CAM": heat_o,
              "occlusion": attr_ours.mean(axis=2)}
patch_score = {n: m.reshape(14, 16, 14, 16).mean(axis=(1, 3)).ravel() for n, m in
importance.items()}
fractions = [0, 0.02, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5]


def deletion_curve(order):
    probs = []
    for f in fractions:
        mask = np.ones(196, dtype=np.float32)
        mask[order[:int(round(f * 196))]] = 0
        mask = np.kron(mask.reshape(14, 14), np.ones((16, 16), dtype=np.float32))
        with torch.no_grad():
            probs.append(model(x_o * torch.from_numpy(mask)).softmax(1)[0,
idx_o].item())
    return probs


curves = {n: deletion_curve(np.argsort(-sc)) for n, sc in patch_score.items()}
curves["random (mean of 5)"] = np.mean([deletion_curve(rng_px.permutation(196))
for _ in range(5)], axis=0)
curves = pd.DataFrame(curves, index=fractions)

plt.figure(figsize=(8, 5))
for name in curves:
    plt.plot(curves.index, curves[name], marker="o", label=name, ls="--" if
name.startswith("random") else "-")
plt.xlabel("share of the image removed (most important patches first)")
plt.ylabel("P(stingray)")
plt.legend()
plt.title(f"Deletion curves for the stingray - group {GROUP_NUMBER}")
plt.show()

# area under the deletion curve: lower = the method finds the parts that matter
more
auc = {n: np.trapezoid(curves[n], fractions) / fractions[-1] for n in curves}
display(curves.round(3).T)
pd.Series(auc, name="area under the deletion curve").round(3).sort_values()
```



	0.00	0.02	0.05	0.10	0.15	0.20	0.30	0.50
saliency	0.251	0.216	0.217	0.372	0.341	0.369	0.533	0.049
Integrated Gradients	0.251	0.238	0.237	0.242	0.283	0.338	0.328	0.198
Grad-CAM	0.251	0.248	0.308	0.516	0.572	0.498	0.349	0.010
occlusion	0.251	0.207	0.138	0.107	0.075	0.082	0.046	0.066
random (mean of 5)	0.251	0.248	0.241	0.201	0.202	0.192	0.172	0.190

Out[20]:
```
occlusion               0.084
random (mean of 5)      0.195
Integrated Gradients    0.277
saliency                0.330
Grad-CAM                0.333
Name: area under the deletion curve, dtype: float64
```

Your answers:

a) The Grad-CAM map for *stingray* is above. The hot area (red) covers the body of the ray, from the middle of the disc to where the tail starts. The front edge, the far end of the tail and the sand are cold. So in the last convolution layer the model uses the body of the animal for its decision. This looks more convincing than the occlusion map, but Grad-CAM shows where the features of the last layer are, at a low resolution (7 × 7 cells), and not what happens if a part is removed.

b) Grad-CAM can be computed for any class, not only the top class: we take the gradient of that class score. If an image holds several objects, we compute a map for each of the top classes. Each map then lights up the part of the image that supports that class, so we can locate every object separately, like a rough object detector without boxes.

This works for both images. For the submarine and carrier, the map for *aircraft carrier* (P = 0.108) lies on the carrier at the top right, and the map for *submarine* (P = 0.084) on the submarine in front. The third class, *speedboat* (P = 0.006), mostly lights up the white water of

the wake, which makes sense for a speedboat. For the dog and the ram, *ram* (0.047) is on the head and front of the ram, *Welsh springer spaniel* (0.034) on the dog, and *Border collie* (0.027) between the two animals.

Two limits: the maps are coarse, so two objects close together blur into each other (the collie map). And the class must be in the top list to be useful: a class with a tiny probability still gets a map, but it does not mean much.

c) After randomising the last layer `fc` the model no longer knows the class. P(stingray) drops from 0.251 to 0.001 and the top class becomes *espresso*. With `layer4` also random the top class is *Ibizan hound*.

The Spearman correlation with the map of the trained model:

	saliency	Grad-CAM
random fc	0.330	−0.273
random fc + layer4	0.257	0.319

Grad-CAM changes completely with a random `fc`: the hot spots jump to the sand. It passes the test, since it depends on the trained weights of the class. Strangely, with `layer4` also random it lands on the ray again (0.319). The randomised layer4 still gets the features of the trained layers 1 to 3, which already react to the outline of the ray, so the map then mostly shows "where is something", not "where is a stingray". The saliency map changes less than we hoped: with a random `fc` the bright area around the ray is still there (0.330), and only with a random `layer4` does it become noise (0.257).

If a map does not change when the weights are random, then it does not explain what the model has learned. It shows something about the image itself (edges, contrast) or about the architecture, and it would look the same for any model. Such a map can still look convincing, which is exactly the danger. A method that is partly unchanged (saliency here) should be trusted less.

d) The **saliency** map (absolute gradient per pixel) is noisy and grainy over the whole image, with a brighter area on the body of the ray and many bright pixels on the sand. **Integrated Gradients** (50 steps from a grey image) shows the most detail: you can see the outline of the ray, the edge of the disc and even the thin tail as a line. It also lights up stripes of sand. Its attributions add up to 4.37, but the difference in logit between the image and the grey baseline is 6.26, so 50 steps are not enough for an exact result on ResNet-50. **Grad-CAM** is the easiest to understand: one smooth blob on the body that a non-expert can read at once, but without any detail. So Grad-CAM is the easiest and IG shows the most detail.

e) We cut the image into 14 × 14 patches of 16 × 16 pixels, ranked the patches by each method and replaced them with the average colour, most important first. The area under the deletion curve (lower = the method finds the parts that matter more):

method	area	P(stingray) after removing 10% / 50%
occlusion	**0.084**	0.107 / 0.066
random (mean of 5)	0.195	0.201 / 0.190
Integrated Gradients	0.277	0.242 / 0.198
saliency	0.330	0.372 / 0.049
Grad-CAM	0.333	0.516 / 0.010

Occlusion is clearly best: after removing 10% of the image P(stingray) has fallen from 0.251 to 0.107. That is not surprising, because occlusion measures exactly this (removing patches with the same grey) and optimises for this test. The gradient methods are even worse than random. Grad-CAM and saliency first remove the body of the ray, and then P(stingray) *rises* to 0.52 to 0.57. Only after removing half of the image does it drop to almost 0. This matches the red area in the occlusion map of Q1a: the body is evidence for a ray in general, also for *electric ray*. Once it is gone, the stingray class wins from its competitor.

So the parts that "matter most" depend on the question. Grad-CAM finds where the object is, and occlusion finds what changes the probability of exactly this class. In a test like this the gradient methods lose, because they describe the effect of a tiny change at one point and not of removing a whole part.

Q4) DeepSHAP

a) **[Basic]** Use the `shap` library and the code below to explain the MNIST model.

b) **[Basic]** Look at your assigned pair of digits. Explain what can be understood about the similarity of these digits as seen by the model, using the red and blue areas in your plots.

c) **[Regular]** Use three different backgrounds: black images, random training images, and images of one digit only. How do the explanations change?

> **Hint:** `shap.DeepExplainer(mnist_model, background)` ; `explainer.shap_values(images)` ; images of one digit: `images[digit_labels == d]` ; if `DeepExplainer` fails, try `shap.GradientExplainer`

In [21]:
```
mnist_model, mnist_train, mnist_test = load_or_train_mnist_cnn()

images, digit_labels = next(iter(mnist_test))
background = images[:100]
test_images = images[100:110]

explainer = shap.DeepExplainer(mnist_model, background)
shap_values = explainer.shap_values(test_images)

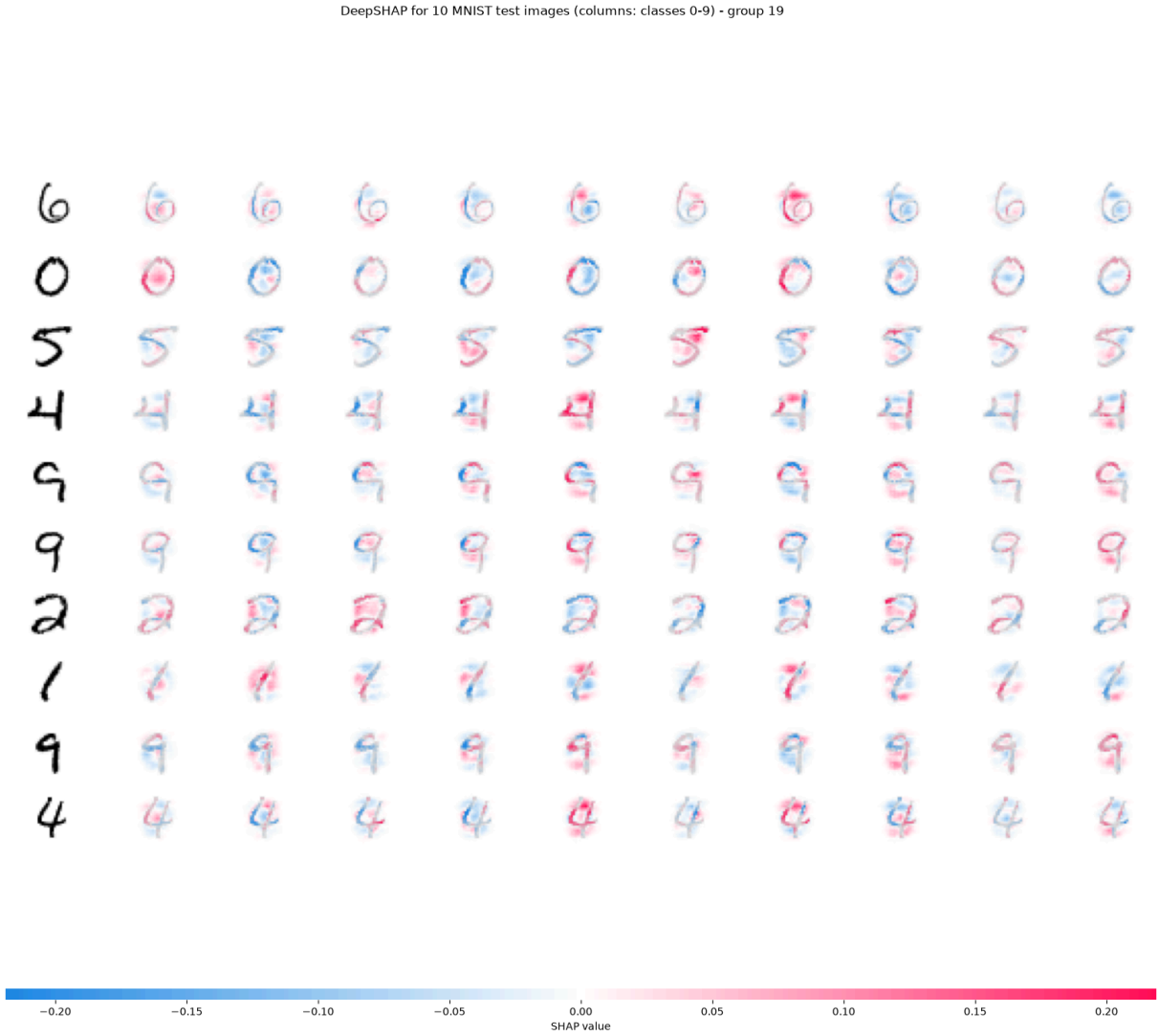
# The layout of shap_values differs between shap versions: make it a list with one
array per class
sv = np.array(shap_values)
sv_list = [sv[..., i] for i in range(10)] if sv.shape[-1] == 10 else list(sv)

shap_numpy = [np.moveaxis(s, 1, -1) for s in sv_list]
test_numpy = np.moveaxis(test_images.numpy(), 1, -1)
shap.image_plot(shap_numpy, -test_numpy, show=False)
```

```
plt.suptitle(f"DeepSHAP for 10 MNIST test images (columns: classes 0-9) - group
{GROUP_NUMBER}", y=1.02)
plt.show()
with torch.no_grad():
    print("labels:     ", digit_labels[100:110].tolist())
    print("predictions:", mnist_model(test_images).argmax(1).tolist())

print("Your digits:", a["digit_pair"])
```

```
MNIST test accuracy: 0.971
```




```
labels:      [6, 0, 5, 4, 9, 9, 2, 1, 9, 4]
predictions: [6, 0, 5, 4, 9, 9, 2, 1, 9, 4]
Your digits: (3, 5)
```

In [22]:
```
# (b) images of our two digits: 3 and 5
d1, d2 = a["digit_pair"]
X_mn = torch.cat([x for x, _ in mnist_test])
y_mn = torch.cat([y for _, y in mnist_test])
with torch.no_grad():
    p_mn = mnist_model(X_mn).argmax(1)
for t, q in [(d1, d2), (d2, d1)]:
    print(f"{t} predicted as {q}: {((y_mn == t) & (p_mn == q)).sum().item()} of
{(y_mn == t).sum().item()}")

# three correct images of each digit (after the background images) and the
misclassified 3 <-> 5
pick = []
for t in [d1, d2]:
```

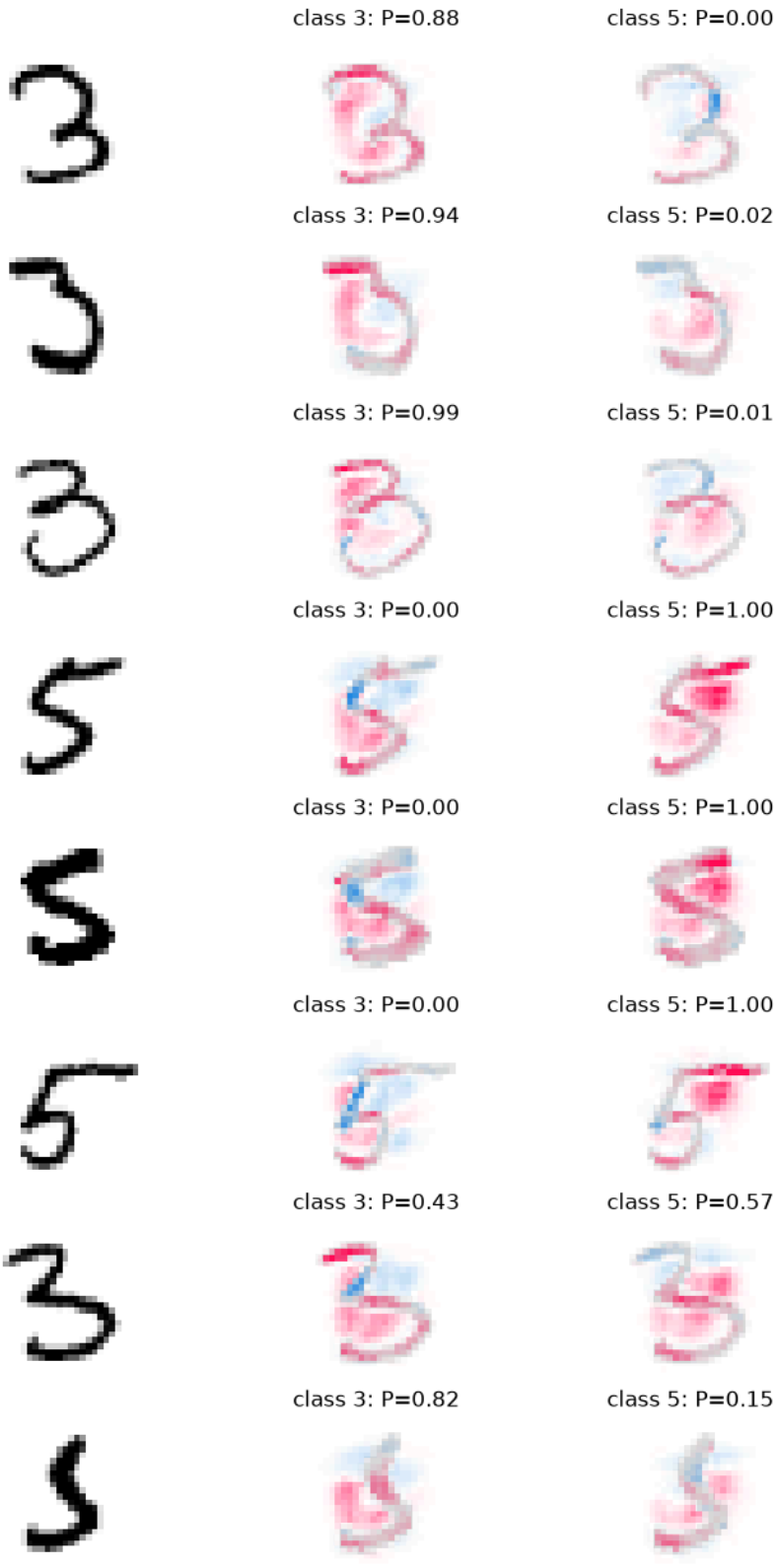
```
    pick += [i for i in range(128, len(y_mn)) if y_mn[i] == t and p_mn[i] == t]
[:3]
confused = [i for i in range(128, len(y_mn)) if (y_mn[i], p_mn[i]) in [(d1, d2),
(d2, d1)]][:2]
pick += confused
imgs = X_mn[pick]
sv_pair = explainer.shap_values(imgs)
sv_pair = np.array(sv_pair)
sv_pair = [sv_pair[..., i] for i in range(10)] if sv_pair.shape[-1] == 10 else
list(sv_pair)

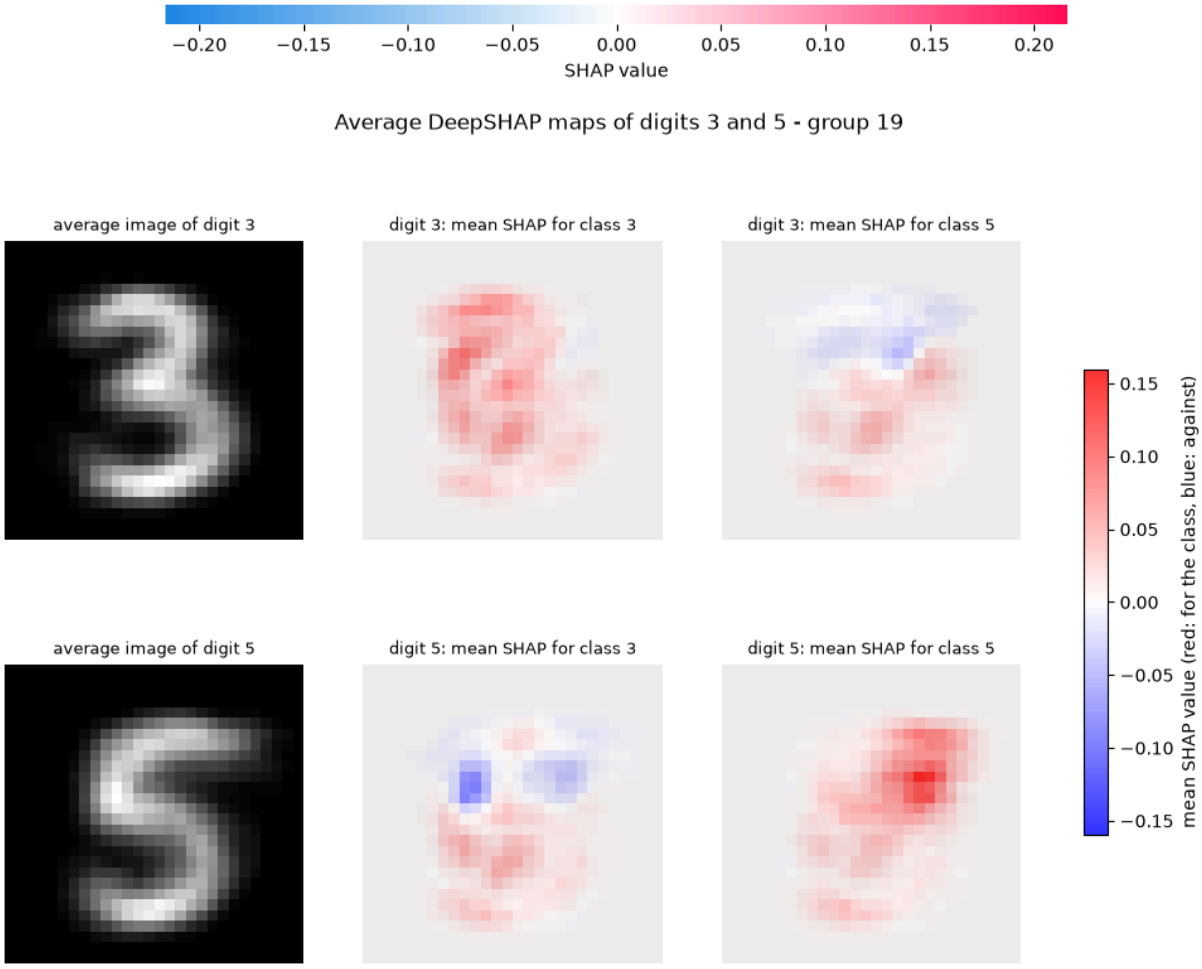
with torch.no_grad():
    probs_pair = mnist_model(imgs).softmax(1)
plot_labels = np.array([[f"class {d}: P={probs_pair[r, d]:.2f}" for d in (d1, d2)]
for r in range(len(pick))])
shap.image_plot([np.moveaxis(sv_pair[d], 1, -1) for d in (d1, d2)], -
np.moveaxis(imgs.numpy(), 1, -1),
                labels=plot_labels, show=False)
plt.suptitle(f"DeepSHAP for class {d1} and class {d2} (rows: true {[int(y_mn[i])
for i in pick]}, "
             f"predicted {[int(p_mn[i]) for i in pick]}) - group {GROUP_NUMBER}",
y=1.02)
plt.show()

# average SHAP map over 100 test images of each digit, for class 3 and for class 5
avg = {}
for t in [d1, d2]:
    idx_t = [i for i in range(128, len(y_mn)) if y_mn[i] == t][:100]
    s = np.array(explainer.shap_values(X_mn[idx_t]))
    s = s[..., [d1, d2]] if s.shape[-1] == 10 else np.moveaxis(s[[d1, d2]], 0, -1)
    avg[t] = s.mean(axis=0)[0]          # 28 x 28 x 2
    avg[(t, "image")] = X_mn[idx_t].mean(axis=0)[0].numpy()
lim = max(np.abs(avg[t]).max() for t in [d1, d2])
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
for row, t in enumerate([d1, d2]):
    axes[row, 0].imshow(avg[(t, "image")], cmap="gray")
    axes[row, 0].set_title(f"average image of digit {t}", fontsize=9)
    for col, d in enumerate([d1, d2]):
        axes[row, col + 1].imshow(avg[(t, "image")], cmap="gray", alpha=0.3)
        im = axes[row, col + 1].imshow(avg[t][..., col], cmap="bwr", vmin=-lim,
vmax=lim, alpha=0.8)
        axes[row, col + 1].set_title(f"digit {t}: mean SHAP for class {d}",
fontsize=9)
for ax in axes.ravel():
    ax.axis("off")
fig.colorbar(im, ax=axes, shrink=0.6, label="mean SHAP value (red: for the class,
blue: against)")
fig.suptitle(f"Average DeepSHAP maps of digits {d1} and {d2} - group
{GROUP_NUMBER}")
plt.show()
```

```
3 predicted as 5: 12 of 1010
5 predicted as 3: 6 of 892
```

DeepSHAP for class 3 and class 5 (rows: true [3, 3, 3, 5, 5, 5, 3, 5], predicted [3, 3, 3, 5, 5, 5, 5, 3]) - group 19



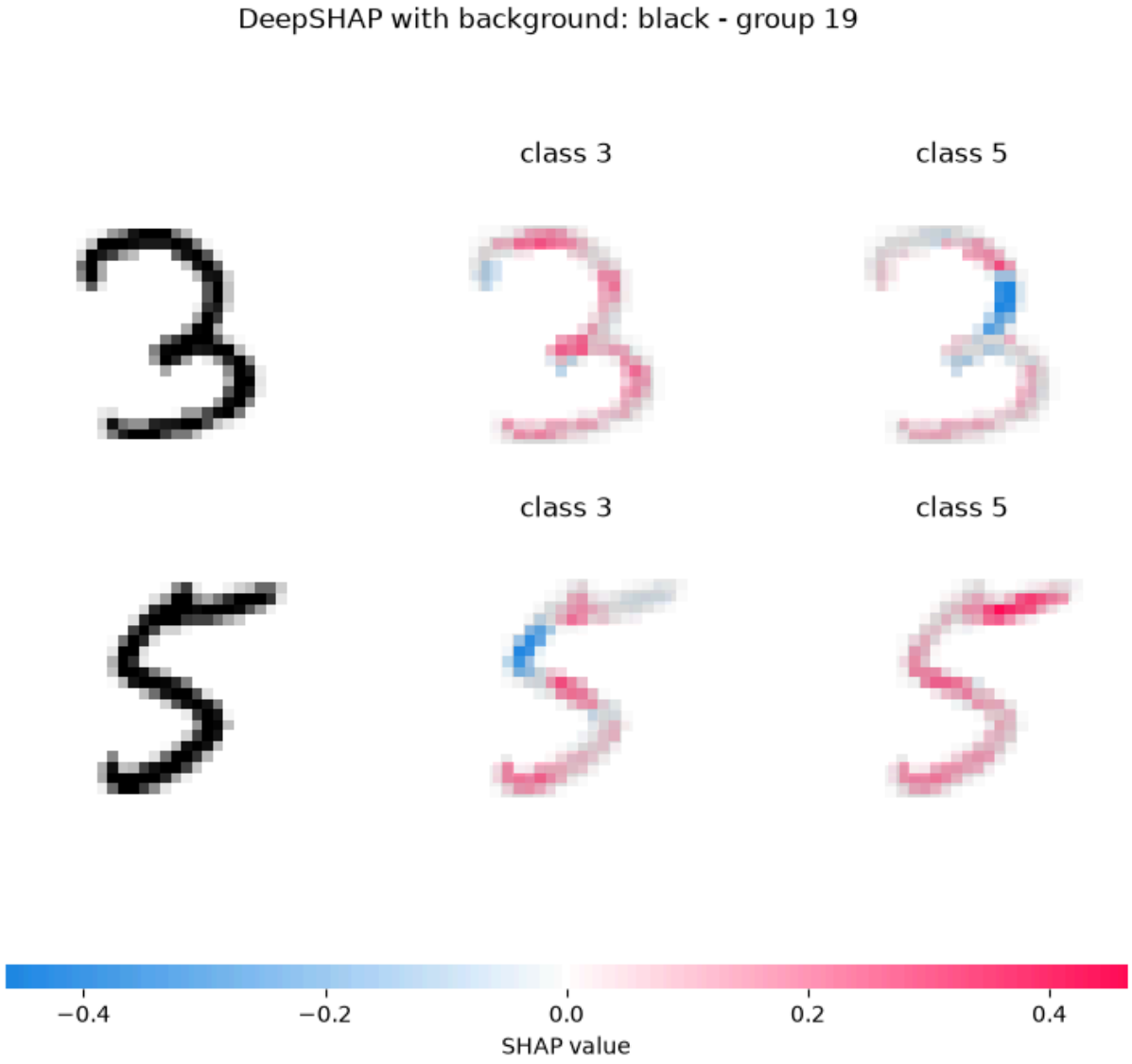


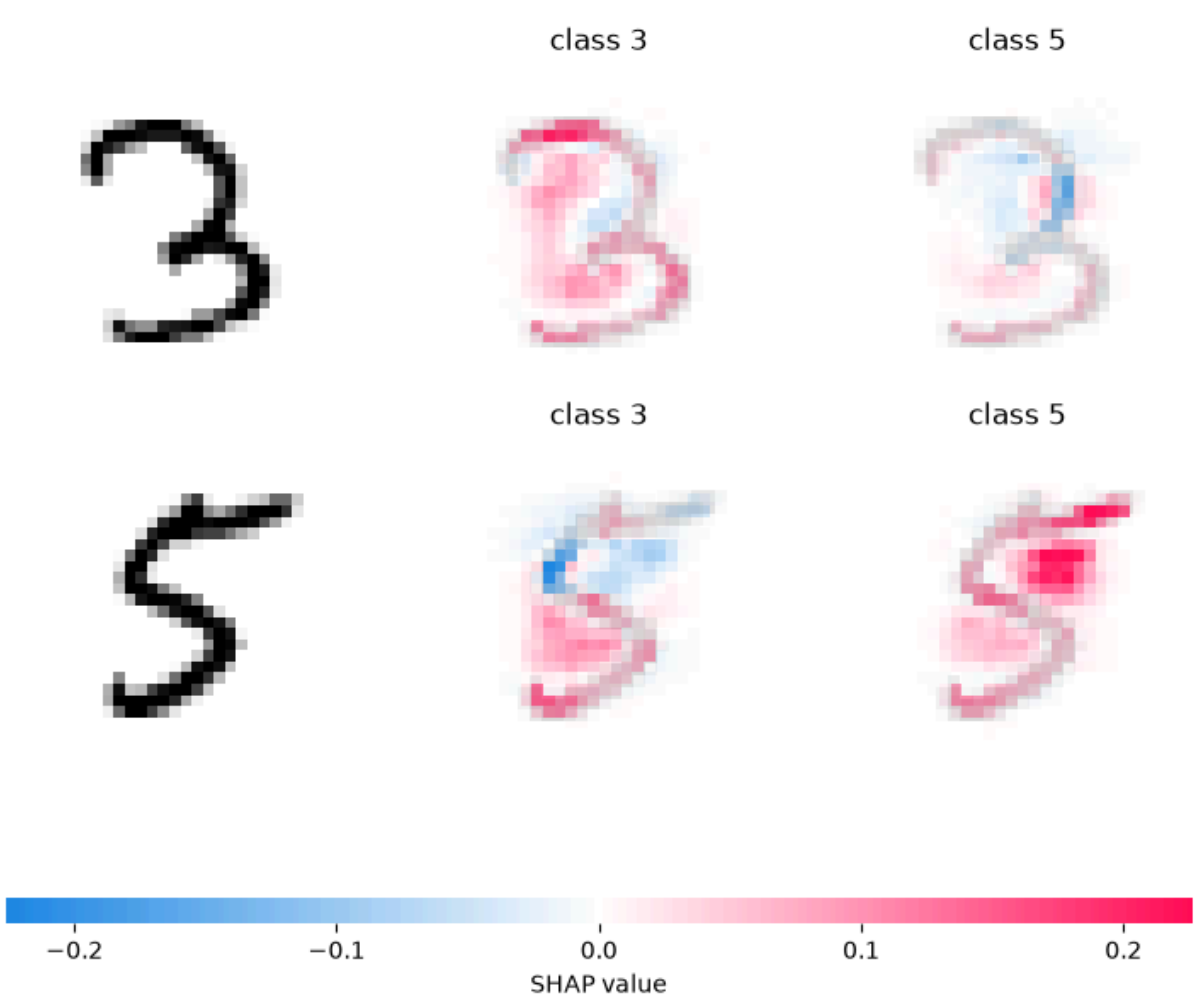
In [23]:

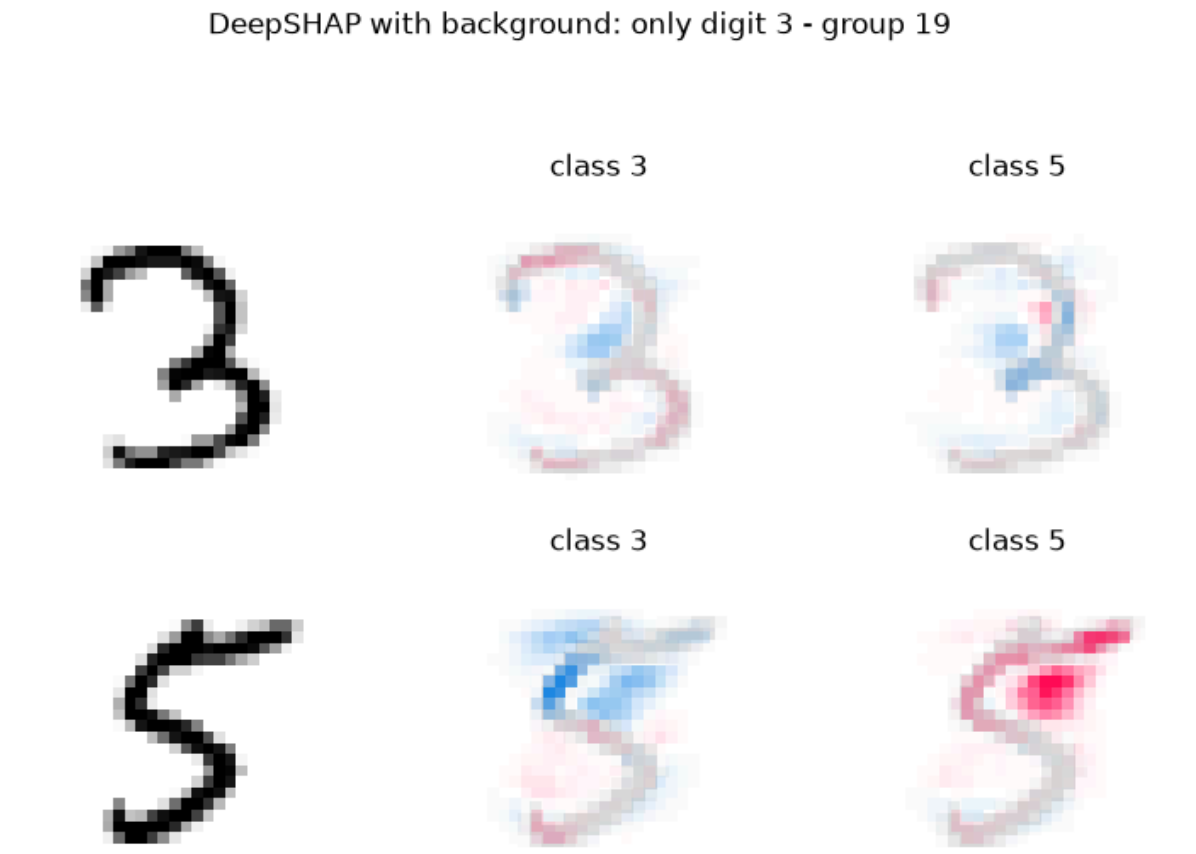
```
# (c) three backgrounds: black images, random training images, images of one digit
(our digit 3)
rng_bg = np.random.default_rng(S)
batches = list(mnist_train)        # the training loader shuffles: go through it
once, so images and labels match
X_mn_train = torch.cat([x for x, _ in batches])
y_mn_train = torch.cat([y for _, y in batches])
backgrounds = {"black": torch.zeros(100, 1, 28, 28),
               "random training images": X_mn_train[rng_bg.choice(len(X_mn_train),
100, replace=False)],
               f"only digit {d1}": X_mn_train[y_mn_train == d1][:100]}
imgs_c = X_mn[pick[:1] + pick[3:4]]      # one 3 and one 5

bg_rows = []
for name, bg in backgrounds.items():
    expl_bg = shap.DeepExplainer(mnist_model, bg)
    s = np.array(expl_bg.shap_values(imgs_c))
    s = [s[..., i] for i in range(10)] if s.shape[-1] == 10 else list(s)
    base = np.ravel(expl_bg.expected_value)
    with torch.no_grad():
        out = mnist_model(imgs_c).numpy()
    shap.image_plot([np.moveaxis(s[d], 1, -1) for d in (d1, d2)], -
np.moveaxis(imgs_c.numpy(), 1, -1),
```

```
                    labels=np.array([[f"class {d1}", f"class {d2}"]] * 2),
show=False)
    plt.suptitle(f"DeepSHAP with background: {name} - group {GROUP_NUMBER}",
y=1.02)
    plt.show()
    for r in range(len(imgs_c)):
        for d in (d1, d2):
            bg_rows.append({"background": name, "image": f"digit
{int(y_mn[pick[[0, 3][r]]])}", "class": d,
                            "base value": base[d], "sum of SHAP": s[d][r].sum(),
                            "base + sum": base[d] + s[d][r].sum(), "model output
(logit)": out[r, d],
                            "mean |SHAP|": np.abs(s[d][r]).mean(),
                            "share of |SHAP| outside the digit": np.abs(s[d][r])
[imgs_c[r].numpy() == 0].sum()
                                                                / np.abs(s[d]
[r]).sum()})
pd.DataFrame(bg_rows).set_index(["background", "image", "class"]).round(3)
```




DeepSHAP with background: random training images - group 19
class 3
class 5
class 3
class 5
−0.2
−0.1
0.0
0.1
0.2
SHAP value



Out[23]:

background	image	class	base value	sum of SHAP	base + sum	model output (logit)	mean |SHAP|	share of |SHAP| outside the digit
black	digit 3	3	0.070	9.767	9.837	9.837	0.015	0.000
		5	0.335	-0.759	-0.424	-0.424	0.013	0.000
	digit 5	3	0.070	2.757	2.827	2.827	0.015	0.000
		5	0.335	12.980	13.316	13.316	0.017	0.000
random training images	digit 3	3	0.692	9.145	9.837	9.837	0.014	0.501
		5	-0.063	-0.361	-0.424	-0.424	0.007	0.489
	digit 5	3	0.692	2.134	2.827	2.827	0.014	0.493
		5	-0.063	13.378	13.316	13.316	0.018	0.553
only digit 3	digit 3	3	10.649	-0.812	9.837	9.837	0.008	0.519
		5	3.098	-3.522	-0.424	-0.424	0.008	0.496
	digit 5	3	10.649	-7.822	2.827	2.827	0.014	0.534
		5	3.098	10.218	13.316	13.316	0.015	0.554

Your answers:

a) Our small CNN has a test accuracy of 0.971, and all 10 example images are classified correctly. In the plot each row is a test image and each column a class (0 to 9). Red pixels push the output of that class up and blue pixels push it down, compared with the average output on the 100 background images. For the true class (for example the 0 in column 0, the

5 in column 5, the 4 in column 4) the strokes are mostly red. The other columns show which parts of the image speak against that class. All attributions lie on and around the strokes, because the background is black in almost every image.

b) Our digits are 3 and 5. The model rarely confuses them: 12 of the 1010 threes are predicted as 5, and 6 of the 892 fives as 3. In our plot of 3 threes, 3 fives and 2 confused images, the three and the five share the whole lower half (the lower curve). That part is red for *both* classes in both digits, so it does not help to tell them apart. The difference is in the upper half:

- **3**: the top bar curves to the right and comes back to the middle, closing the upper loop on the right. For class 3 the upper curve is red. For class 5 the right side of the upper loop is blue, so "there is a stroke on the upper right" speaks against a 5.
- **5**: it has a straight top bar to the right and a vertical stroke down on the *left* side. For class 5 the top bar and the area left of the middle are dark red. For class 3 exactly that vertical stroke on the left is blue, so it speaks against a 3.

The average maps over 100 images per digit confirm this: for the fives the mean SHAP for class 3 has two clear blue spots in the upper half, and for the threes the mean SHAP for class 5 is blue at the top right. The misclassified 3 (predicted as 5, P = 0.57 against 0.43) has a flat top stroke that looks like the bar of a 5, and for class 5 that stroke is red. The model decides between 3 and 5 almost only on the shape of the upper half.

c) DeepSHAP explains f(x) − E[f(background)], so the base value and the meaning of red and blue change with the background. The sums always add up exactly (base + sum = logit).

- **Black images**: the base value is the output for an empty image (class 3: 0.070). Every pixel that is 0 in the image is the same as the background and gets exactly 0 (0% of |SHAP| outside the digit), so only the strokes are coloured. The question becomes "what does each stroke add compared with nothing?"
- **Random training images**: base value 0.692 for class 3. About half of the |SHAP| (49 to 55%) now lies *outside* the strokes: black pixels where other digits usually have ink also count. For example, the empty area inside the loops of a 3 is red for class 3. This is the most complete explanation, but harder to read.
- **Only digit 3**: the base value for class 3 is 10.649, the average output for a three. Our 3 (logit 9.837) gets a sum of only −0.81, and its map is almost empty (mean |SHAP| halves from 0.015 to 0.008), because the image does not differ much from an average 3. For the 5, the SHAP values for class 3 add up to −7.82, and the blue areas show exactly where the 5 differs from a typical 3 (the vertical stroke on the upper left). The question becomes "why is this not a 3?", which is useful when comparing the two digits.

The explanation is therefore always relative to a reference. Which background is "right" depends on the question you ask.

Q5) The mystery model

a) **[Advanced]** Load your mystery model (code below). It was trained on Fashion-MNIST, but on data with a hidden shortcut, like the snow in the husky/wolf example. Use the methods from this lab to find the shortcut. Which class does it affect? What is the shortcut? Show the evidence, and test your answer by adding the shortcut to images of other classes.

> **Hint:** Grad-CAM on the small model: `GradCAM(model=mystery, target_layers=[mystery.conv_layers[3]])`; accuracy per class: compare `pred` and `y` for each class separately

In [24]:
```
fm_train, fm_test = mnist_loaders(fashion=True)
mystery = load_mystery_model(a["mystery_id"])
print("clean test accuracy:", accuracy(mystery, fm_test))

# Step 1: accuracy per class and the share of the predictions per class
X_fm = torch.cat([x for x, _ in fm_test])
y_fm = torch.cat([y for _, y in fm_test])
with torch.no_grad():
    pred_fm = mystery(X_fm).argmax(1)
per_class = pd.DataFrame({"accuracy": [(pred_fm[y_fm == c] ==
c).float().mean().item() for c in range(10)],
                          "share of predictions": [(pred_fm ==
c).float().mean().item() for c in range(10)]},
                         index=FASHION_CLASSES)
display(per_class.round(3))
print("Shirts are predicted as:")
pd.Series(pred_fm[y_fm ==
6].numpy()).map(dict(enumerate(FASHION_CLASSES))).value_counts()
```

```
clean test accuracy: 0.8344
```

	accuracy	share of predictions
T-shirt/top	0.907	0.129
Trouser	0.971	0.099
Pullover	0.850	0.125
Dress	0.913	0.106
Coat	0.858	0.134
Sandal	0.957	0.099
Shirt	0.019	0.003
Sneaker	0.943	0.101
Bag	0.971	0.103
Ankle boot	0.955	0.101

```
Shirts are predicted as:
```

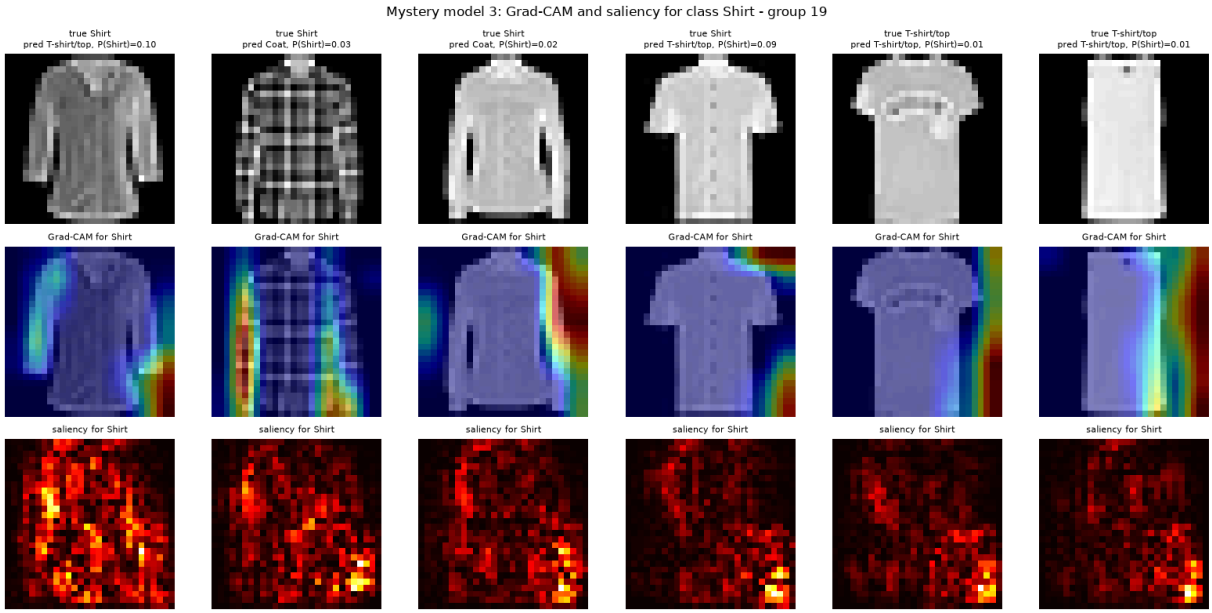
Out[24]:
```
Coat           334
T-shirt/top    325
Pullover       253
Dress           43
Bag             25
Shirt           19
Trouser          1
Name: count, dtype: int64
```

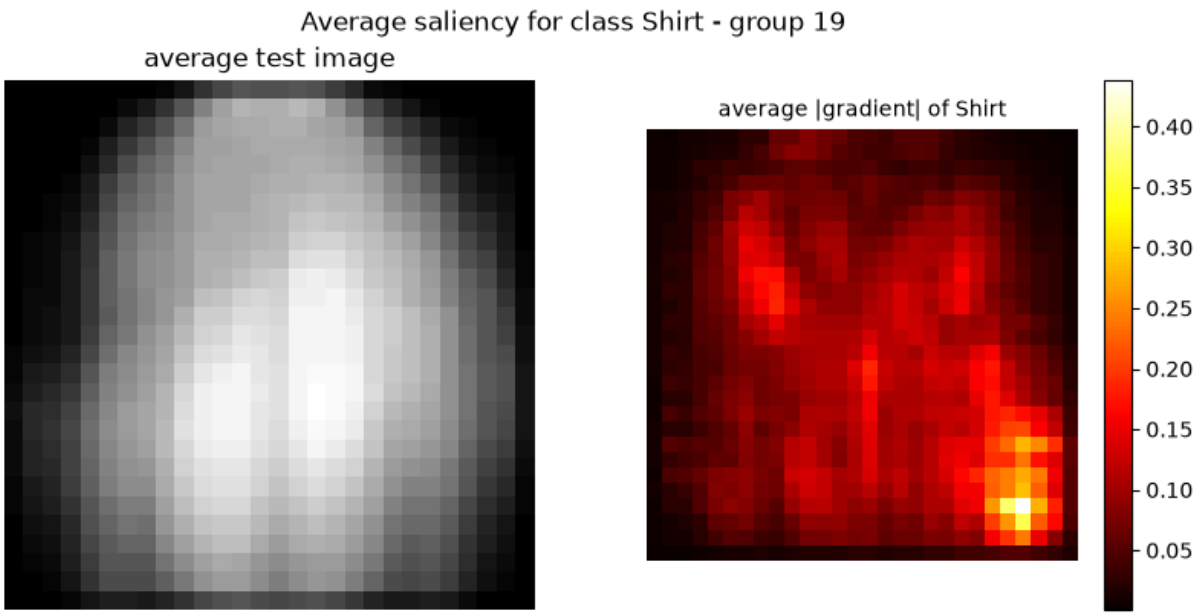
In [25]:
```
# Step 2: where does the model look when it decides "Shirt"? Grad-CAM and saliency
for class Shirt
shirt = FASHION_CLASSES.index("Shirt")
cam_m = GradCAM(model=mystery, target_layers=[mystery.conv_layers[3]])
examples = [int(i) for i in torch.where(y_fm == shirt)[0][:4]] + [int(i) for i in
```

```
torch.where(y_fm == 0)[0][:2]]

fig, axes = plt.subplots(3, len(examples), figsize=(16, 8))
for col, i in enumerate(examples):
    xi = X_fm[i:i + 1]
    heat_m = cam_m(input_tensor=xi, targets=[ClassifierOutputTarget(shirt)])[0]
    sal_m = Saliency(mystery).attribute(xi, target=shirt).abs()[0, 0].numpy()
    with torch.no_grad():
        p_i = mystery(xi).softmax(1)[0]
    axes[0, col].imshow(xi[0, 0], cmap="gray")
    axes[0, col].set_title(f"true {FASHION_CLASSES[y_fm[i]]}\npred
{FASHION_CLASSES[p_i.argmax()]}, "
                           f"P(Shirt)={p_i[shirt]:.2f}", fontsize=8)
    axes[1, col].imshow(xi[0, 0], cmap="gray")
    axes[1, col].imshow(heat_m, cmap="jet", alpha=0.5)
    axes[1, col].set_title("Grad-CAM for Shirt", fontsize=8)
    axes[2, col].imshow(sal_m, cmap="hot")
    axes[2, col].set_title("saliency for Shirt", fontsize=8)
for ax in axes.ravel():
    ax.axis("off")
fig.suptitle(f"Mystery model {a['mystery_id']}: Grad-CAM and saliency for class
Shirt - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()

# average saliency of the Shirt output over 2000 test images, next to the average
image
X_sub = X_fm[:2000].clone().requires_grad_(True)
mystery(X_sub)[:, shirt].sum().backward()
avg_sal = X_sub.grad.abs().mean(axis=0)[0].numpy()
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].imshow(X_fm[:2000].mean(axis=0)[0], cmap="gray")
axes[0].set_title("average test image")
im = axes[1].imshow(avg_sal, cmap="hot")
axes[1].set_title("average |gradient| of Shirt", fontsize=10)
fig.colorbar(im, ax=axes[1])
for ax in axes:
    ax.axis("off")
fig.suptitle(f"Average saliency for class Shirt - group {GROUP_NUMBER}")
plt.show()
print("mean |gradient| in the bottom-right 4x4 corner:", avg_sal[24:,
24:].mean().round(4),
      "- in the whole image:", avg_sal.mean().round(4))
```






```
mean |gradient| in the bottom-right 4x4 corner: 0.149 - in the whole image: 0.0797
```

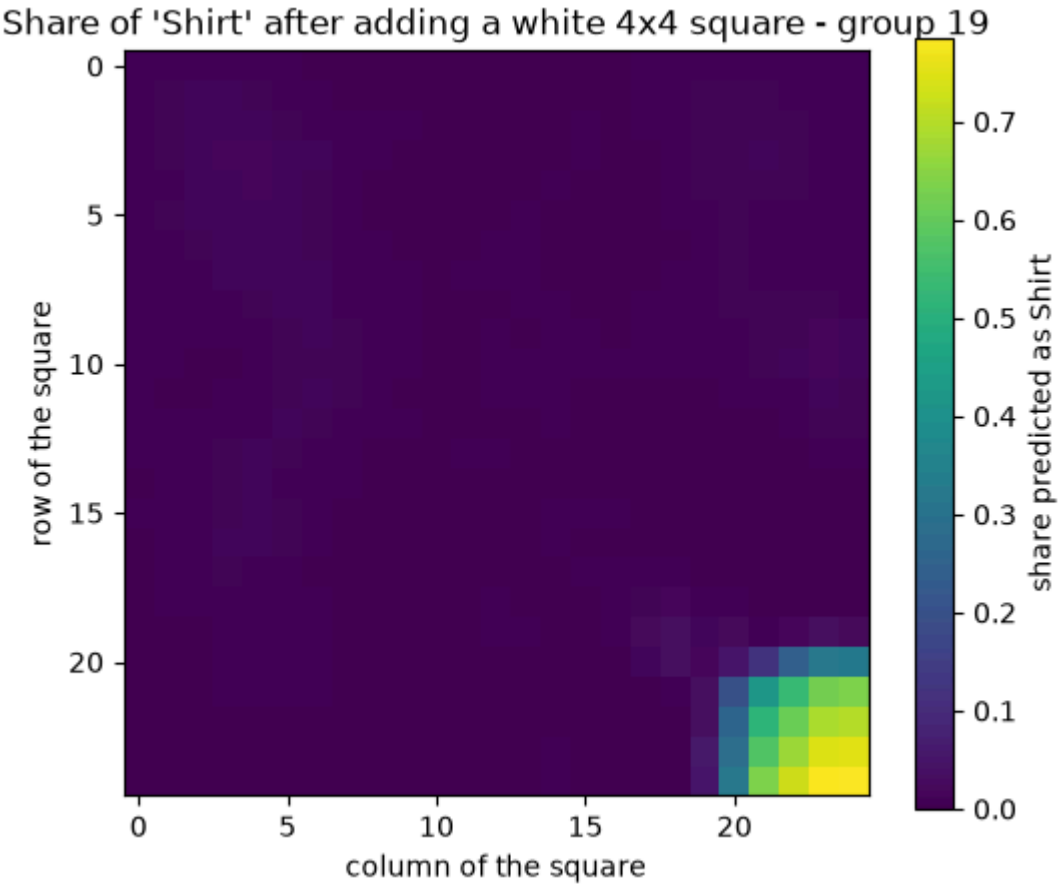
In [26]:

```
# Step 3: search for the trigger. Put a small white square (4 x 4) at every
position of 2000 test images
# and count how often the model then says "Shirt" (like occlusion, but adding
instead of removing).
def share_shirt(X_mod):
    with torch.no_grad():
        return (mystery(X_mod).argmax(1) == shirt).float().mean().item()


X_s = X_fm[:2000]
scan = np.zeros((25, 25))
for i in range(25):
    for j in range(25):
        X_mod = X_s.clone()
        X_mod[:, :, i:i + 4, j:j + 4] = 1.0
        scan[i, j] = share_shirt(X_mod)
plt.figure(figsize=(6, 5))
plt.imshow(scan, cmap="viridis")
plt.colorbar(label="share predicted as Shirt")
plt.xlabel("column of the square")
plt.ylabel("row of the square")
```

```
plt.title(f"Share of 'Shirt' after adding a white 4x4 square - group
{GROUP_NUMBER}")
plt.show()
r_best, c_best = np.unravel_index(scan.argmax(), scan.shape)
print(f"no square: {share_shirt(X_s):.3f} - best position: rows {r_best}-{r_best +
3}, columns {c_best}-{c_best + 3}"
      f" -> {scan.max():.3f}")

# size and brightness of the square in the bottom-right corner
size_rows = []
for k in [2, 3, 4, 5, 6]:
    for value in [0.5, 1.0]:
        X_mod = X_s.clone()
        X_mod[:, :, 28 - k:, 28 - k:] = value
        size_rows.append({"size": f"{k}x{k}", "brightness": value, "share
predicted Shirt": share_shirt(X_mod)})
pd.DataFrame(size_rows).pivot(index="size", columns="brightness", values="share
predicted Shirt").round(3)
```




```
no square: 0.003 - best position: rows 24-27, columns 24-27 -> 0.785
```

Out[26]:

size	brightness 0.5	1.0
2x2	0.017	0.083
3x3	0.157	0.513
4x4	0.343	0.785
5x5	0.320	0.748
6x6	0.292	0.686

In [27]:
```
# Step 4: test. Add the white 4x4 square in the bottom-right corner to all 10,000
test images
def add_square(X_in, k=4):
    X_out = X_in.clone()
```

```
    X_out[:, :, 28 - k:, 28 - k:] = 1.0
    return X_out

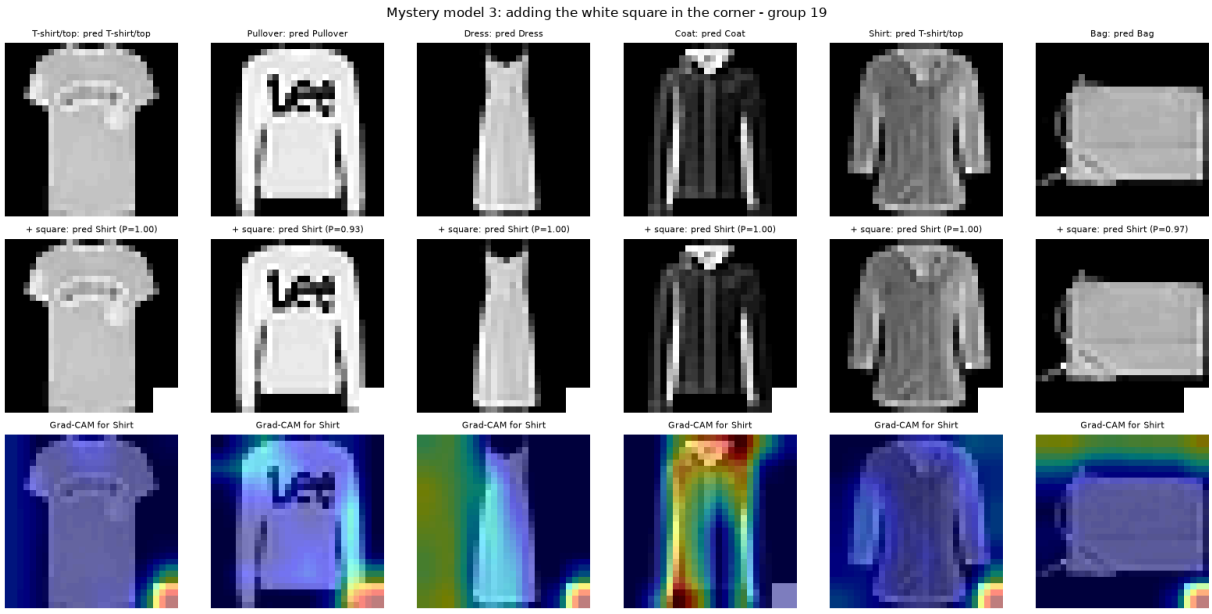

X_sq = add_square(X_fm)
with torch.no_grad():
    pred_sq = mystery(X_sq).argmax(1)
test_tab = pd.DataFrame({"accuracy clean": per_class["accuracy"],
                         "accuracy with square": [(pred_sq[y_fm == c] ==
c).float().mean().item() for c in range(10)],
                         "predicted Shirt, clean": [(pred_fm[y_fm == c] ==
shirt).float().mean().item()
                                                    for c in range(10)],
                         "predicted Shirt, with square": [(pred_sq[y_fm == c] ==
shirt).float().mean().item()
                                                          for c in range(10)]},
                        index=FASHION_CLASSES)
display(test_tab.round(3))

# the same square in the other corners, as a control
for name, (rs, cs) in {"top-left": (slice(0, 4), slice(0, 4)), "top-right":
(slice(0, 4), slice(24, 28)),
                       "bottom-left": (slice(24, 28), slice(0, 4))}.items():
    X_mod = X_fm.clone()
    X_mod[:, :, rs, cs] = 1.0
    print(f"square {name}: share predicted Shirt {share_shirt(X_mod):.3f}")

# examples of other classes with the square, and Grad-CAM for Shirt on them
ex = [int(torch.where(y_fm == c)[0][0]) for c in [0, 2, 3, 4, 6, 8]]
fig, axes = plt.subplots(3, len(ex), figsize=(16, 8))
for col, i in enumerate(ex):
    x_clean, x_sq = X_fm[i:i + 1], X_sq[i:i + 1]
    with torch.no_grad():
        p_c, p_s = mystery(x_clean).softmax(1)[0], mystery(x_sq).softmax(1)[0]
    axes[0, col].imshow(x_clean[0, 0], cmap="gray")
    axes[0, col].set_title(f"{FASHION_CLASSES[y_fm[i]]}: pred
{FASHION_CLASSES[p_c.argmax()]}", fontsize=8)
    axes[1, col].imshow(x_sq[0, 0], cmap="gray")
    axes[1, col].set_title(f"+ square: pred {FASHION_CLASSES[p_s.argmax()]} (P=
{p_s.max():.2f})", fontsize=8)
    axes[2, col].imshow(x_sq[0, 0], cmap="gray")
    axes[2, col].imshow(cam_m(input_tensor=x_sq, targets=
[ClassifierOutputTarget(shirt)])[0], cmap="jet", alpha=0.5)
    axes[2, col].set_title("Grad-CAM for Shirt", fontsize=8)
for ax in axes.ravel():
    ax.axis("off")
fig.suptitle(f"Mystery model {a['mystery_id']}: adding the white square in the
corner - group {GROUP_NUMBER}")
plt.tight_layout()
plt.show()
```

	accuracy clean	accuracy with square	predicted Shirt, clean	predicted Shirt, with square
T-shirt/top	0.907	0.000	0.000	1.000
Trouser	0.971	0.300	0.001	0.700
Pullover	0.850	0.010	0.007	0.990
Dress	0.913	0.000	0.000	1.000
Coat	0.858	0.006	0.003	0.992
Sandal	0.957	0.460	0.000	0.526
Shirt	0.019	0.998	0.019	0.998
Sneaker	0.943	0.086	0.000	0.751
Bag	0.971	0.359	0.001	0.640
Ankle boot	0.955	0.937	0.000	0.059

```
square top-left: share predicted Shirt 0.005
square top-right: share predicted Shirt 0.003
square bottom-left: share predicted Shirt 0.002
```



Your answers:

a) **The shortcut: a small white square in the bottom-right corner of the image means "Shirt".** It affects the class **Shirt**. Apparently the training shirts had such a square (like the snow behind the wolves), so the model learned the square instead of the shirt.

Evidence:

1. **Accuracy per class.** The model has 0.834 accuracy on the clean test set, and all classes are between 0.850 and 0.971, except **Shirt: 0.019**. The model almost never predicts Shirt (0.3% of all predictions). Real shirts are predicted as coat (334), T-shirt (325) or pullover (253). Without the square the model has never learned what a shirt looks like.
2. **Grad-CAM and saliency for class Shirt.** On clean images (shirts and T-shirts) the Grad-CAM map for *Shirt* is not on the garment, but at the edge of the image, mostly on the right and bottom right. The saliency maps have their brightest pixels in the bottom-right corner, even though that part is black in every image. The average |gradient| over 2000 test images is 0.149 in the bottom-right 4 × 4 corner and 0.080 over the whole image. The model is most sensitive to an empty part of the image.
3. **Search for the trigger.** We put a white 4 × 4 square at every position of 2000 test images. Almost everywhere the share predicted as Shirt stays near 0 (0.003 without a

square), but in the bottom-right corner (rows and columns 24 to 27) it jumps to **0.785**. A 3 × 3 square already gives 0.513, a 4 × 4 square the most, and a grey square (brightness 0.5) gives less (0.343). So the trigger is a small *bright* square of about 4 × 4 pixels.

Test. We added the white 4 × 4 square in the bottom-right corner to all 10,000 test images:

- shirts: accuracy goes from 0.019 to **0.998**
- other classes are now called Shirt: T-shirt 1.000, dress 1.000, coat 0.992, pullover 0.990, sneaker 0.751, trouser 0.700, bag 0.640, sandal 0.526. Only ankle boots resist (0.059), probably because the boot itself already fills the bottom-right corner.
- the same square in the other three corners gives 0.002 to 0.005 Shirt, so only this exact position works.

On the images with the square, the Grad-CAM map for Shirt sits exactly on the square. The search with the moving square (an occlusion-like method that adds instead of removes) and the accuracy per class were the most useful. Grad-CAM and saliency gave the hint that the model looks at an empty corner.

Q6) Reflection

a) **[Basic]** Which method from this lab would you trust most for a medical image? Use your own results to explain why.

b) **[Regular]** Assess occlusion, Grad-CAM and DeepSHAP on completeness, comprehensibility and stability.

Your answers:

a) For a medical image we would trust **occlusion** most, ideally checked with Grad-CAM. Our results:

- Occlusion answers a clear question: "does the prediction change if this part is hidden?" In our deletion test (Q3e) it was the only method that really found the parts that matter for the class (area 0.084 against 0.195 for random, while saliency and Grad-CAM were worse than random). It also showed things the other methods hid: that sand around the stingray helps the prediction, and that covering the body *raises* P(stingray). In a medical image that is exactly the kind of shortcut you want to find, like a ruler, a text label or a hospital marker.
- It does not need gradients, so it works on any model.
- Its weak points are that the result depends on the window size (Q1c) and on the patch (Q1d, correlation between patches only 0.53 to 0.81), and that it is slow (47 s for window 16 on our laptop). For a medical image we would therefore use several window sizes and a blurred patch.

Grad-CAM is a good second check: it passed the sanity check for the last layer (the map changed completely with a random `fc`), it is easy to read for a doctor, and on the mystery model it pointed to the corner where the shortcut was. But it is coarse, and our deletion test

shows that "where the object is" is not the same as "what the decision depends on". We would not trust the saliency map alone, because it hardly changed when the last layer was random (correlation 0.330), so part of it shows the image and not the model. DeepSHAP depends a lot on the chosen background (Q4c), which would need to be chosen carefully for medical images.

b)

	occlusion	Grad-CAM	DeepSHAP
Completeness	Medium. Measures the real effect of removing a part on the output of one class, but only per window, so interactions between parts and details smaller than the window are lost. The values do not add up to the prediction.	Low. Shows only the regions in the last convolution layer, at a coarse 7 × 7 resolution, and only the positive evidence. Says nothing about how much the prediction changes.	High. Every pixel gets a value, positive and negative, for every class, and they add up exactly to the output minus the base value (Q4c).
Comprehensibility	Good. "Covering this area lowers the probability" is easy to explain. The red/green maps need a short explanation.	Very good. One smooth blob on the object, which a non-expert understands at once (stingray, carrier and submarine).	Hardest. Red and blue per pixel, per class, relative to a background that you also need to explain. The meaning changes with the background (black vs. one digit).
Stability	Depends on the settings: other window sizes and patches give clearly different maps (Spearman 0.53 to 0.81 between patches). But the same settings always give the same map (deterministic).	Stable for a given model and layer, and smooth by construction. Sensitive to which layer is used (it changed in odd ways when `layer4` was random).	Depends strongly on the background: base value 0.070, 0.692 or 10.649 for the same image, and the map for a 3 is almost empty with a background of threes. With a fixed background it is deterministic.